

CONVOLUTIONAL NEURAL NETWORKS

OSCAR LLORENTE GONZALEZ

OLLLORENTE@COMILLAS.EDU

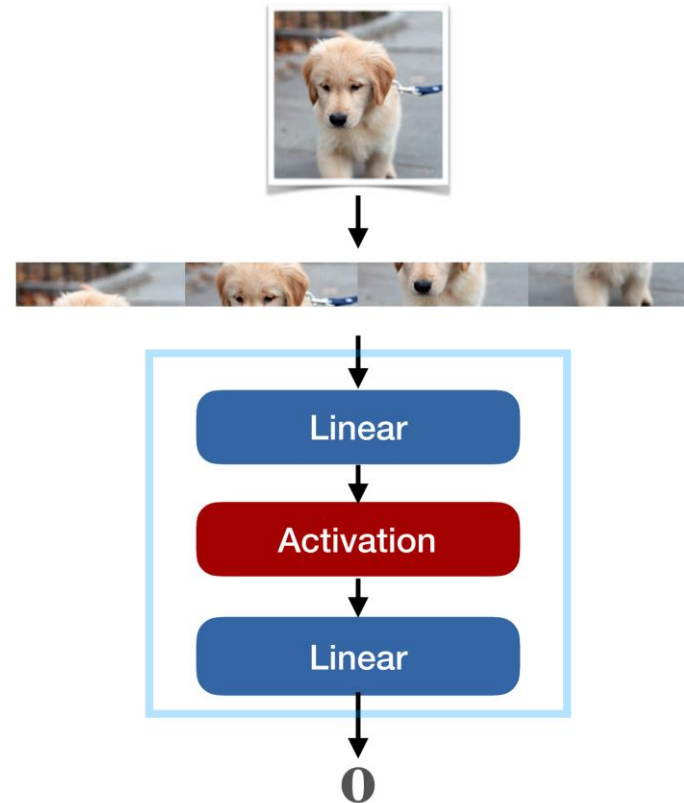
INDEX

- 1 - Introduction
- 2 - Convolutional Layers
- 3 - Pooling
- 4 - Padding, Striding and Groups
- 5 - Design Principles of Convolutional Neural Networks
- 6 - Model Examples
- 7 - Depth
- 8 - Up-Convolutions
- 9 - Other Dimensions
- 10 - Fold & Unfold
- 11 - Gather & Scatter



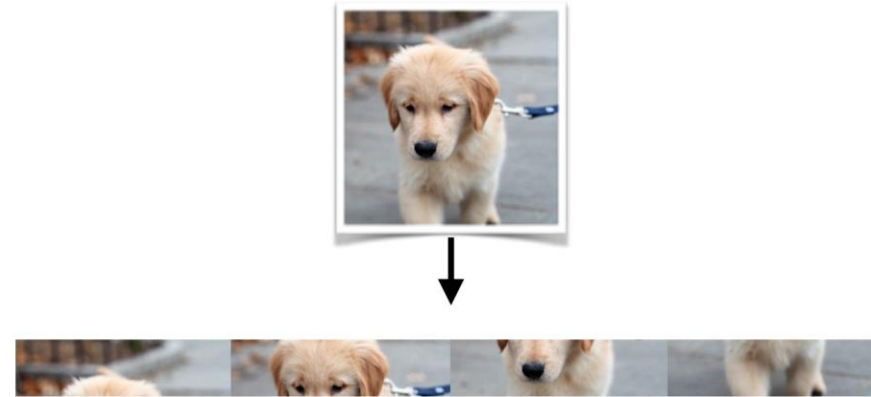
1 - I N T R O D U C T I O N

PROBLEMS WITH FEEDFORWARD NETS

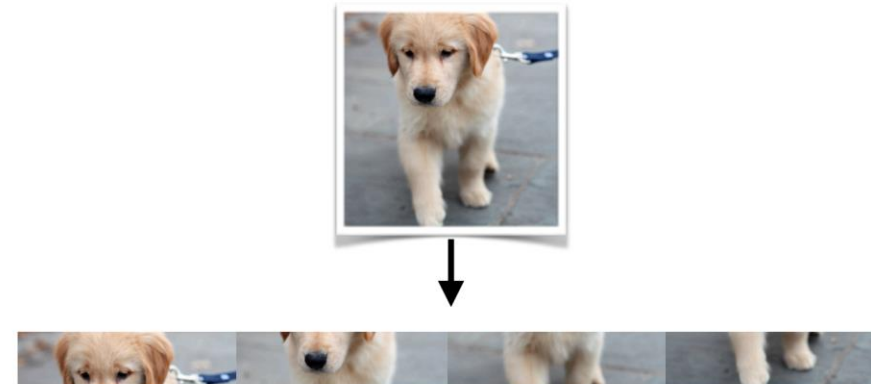


PROBLEMS WITH FEEDFORWARD NETS

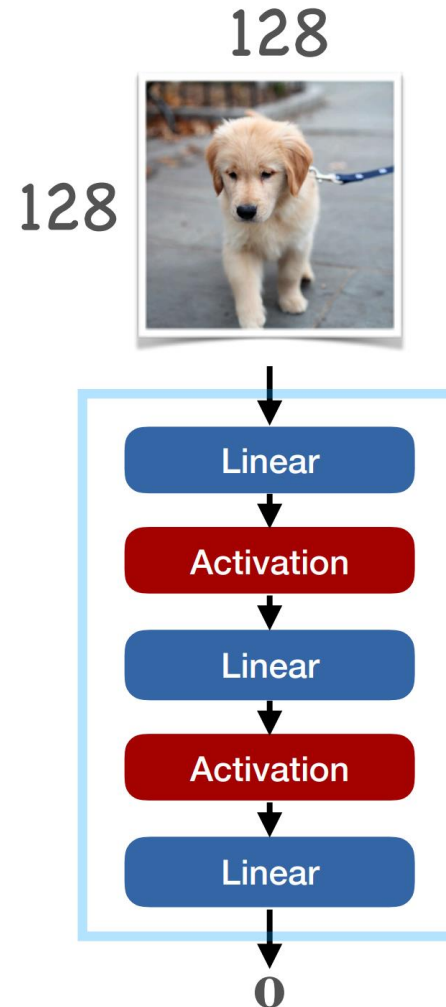
Visual patterns are
shift, rotation and scale
invariant



Fully connected
networks are not

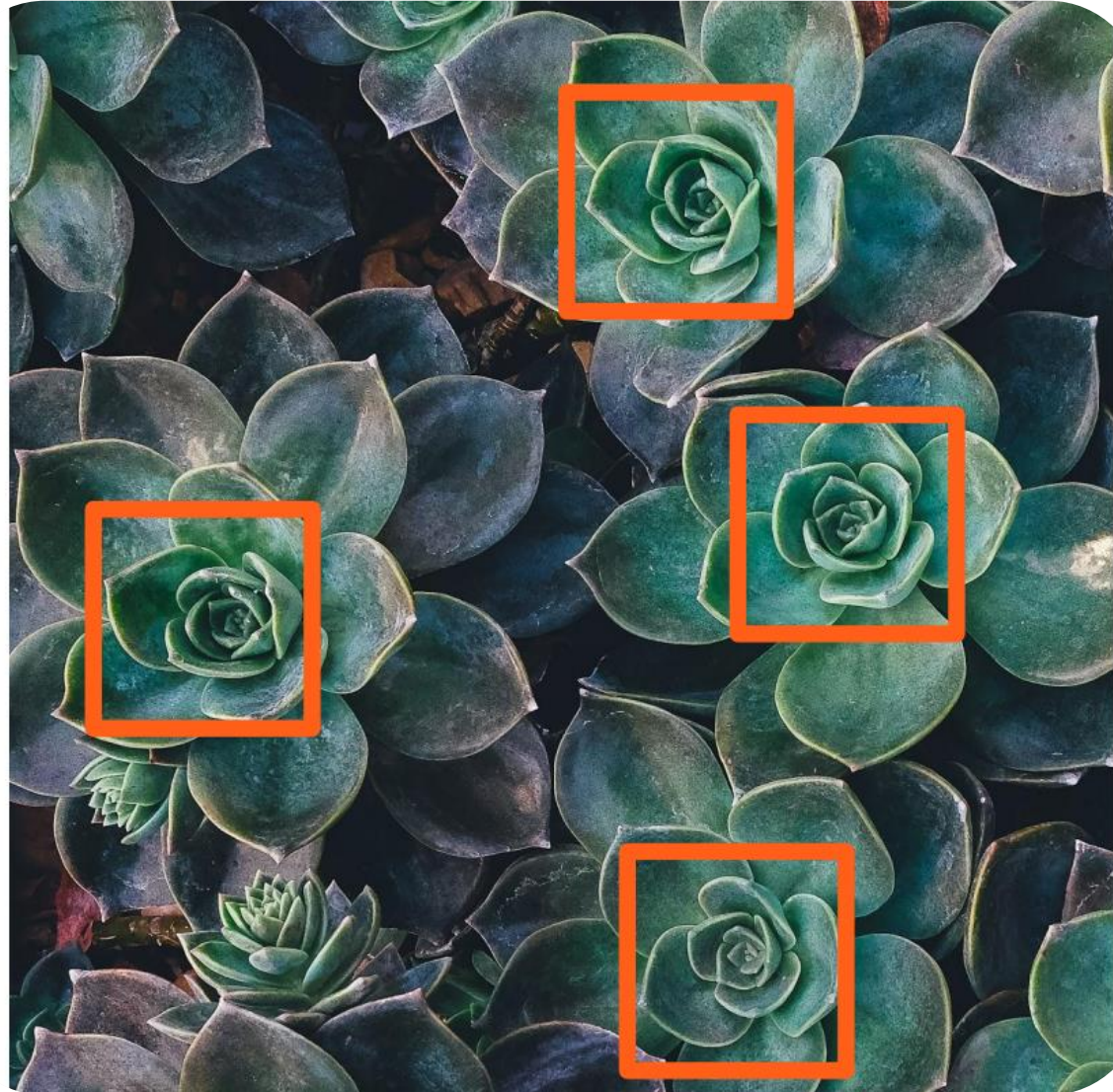


PROBLEMS WITH FEEDFORWARD NETS

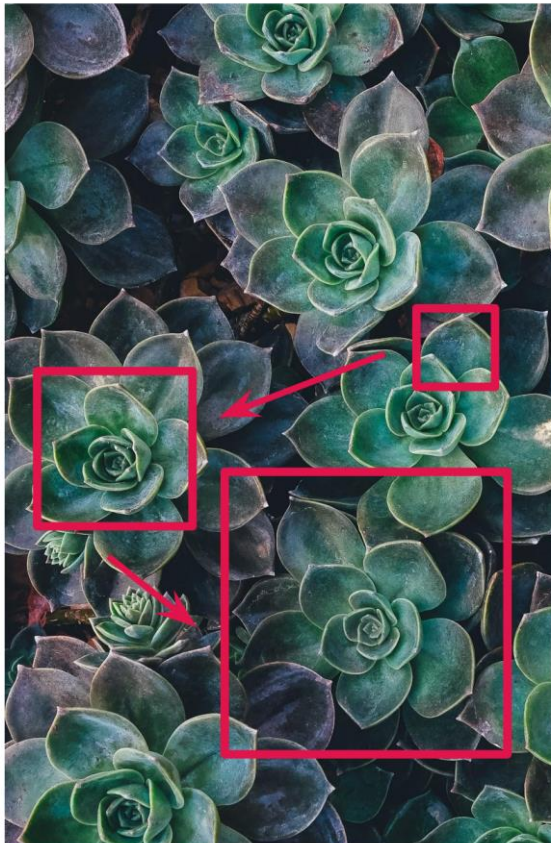


TAKING ADVANTAGE OF TOPOLOGICAL STRUCTURE

Weight sharing: use the same network parameters to detect local patterns at many locations in the image



TAKING ADVANTAGE OF TOPOLOGICAL STRUCTURE



Hierarchy: local low-level features
are composed into larger, more
abstract features



edges and textures



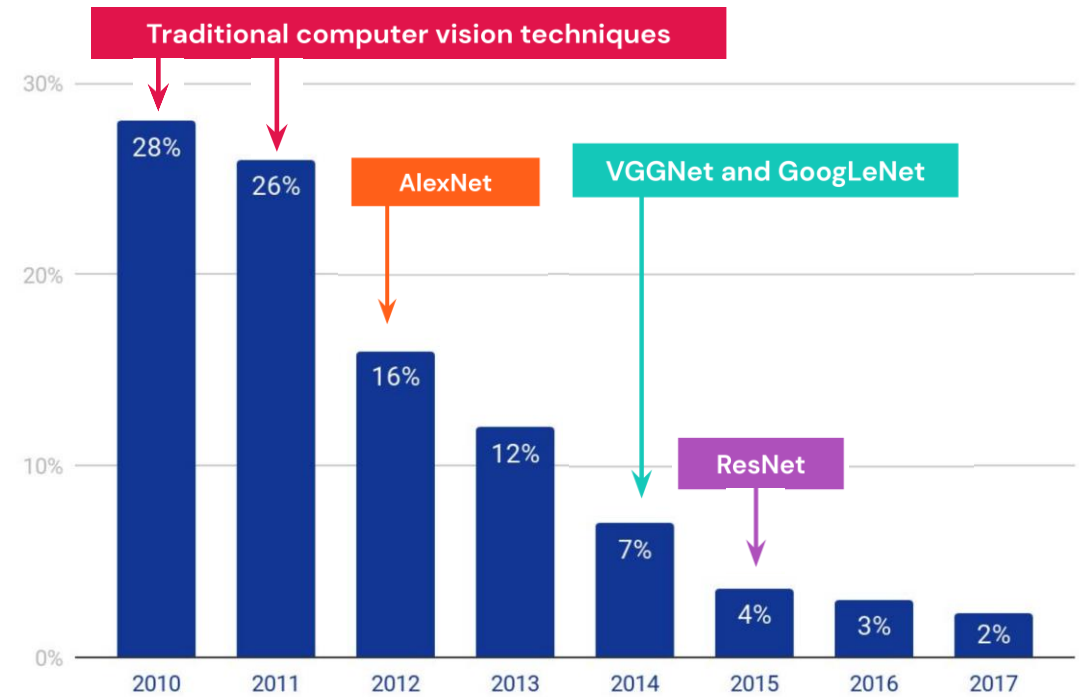
object parts



objects

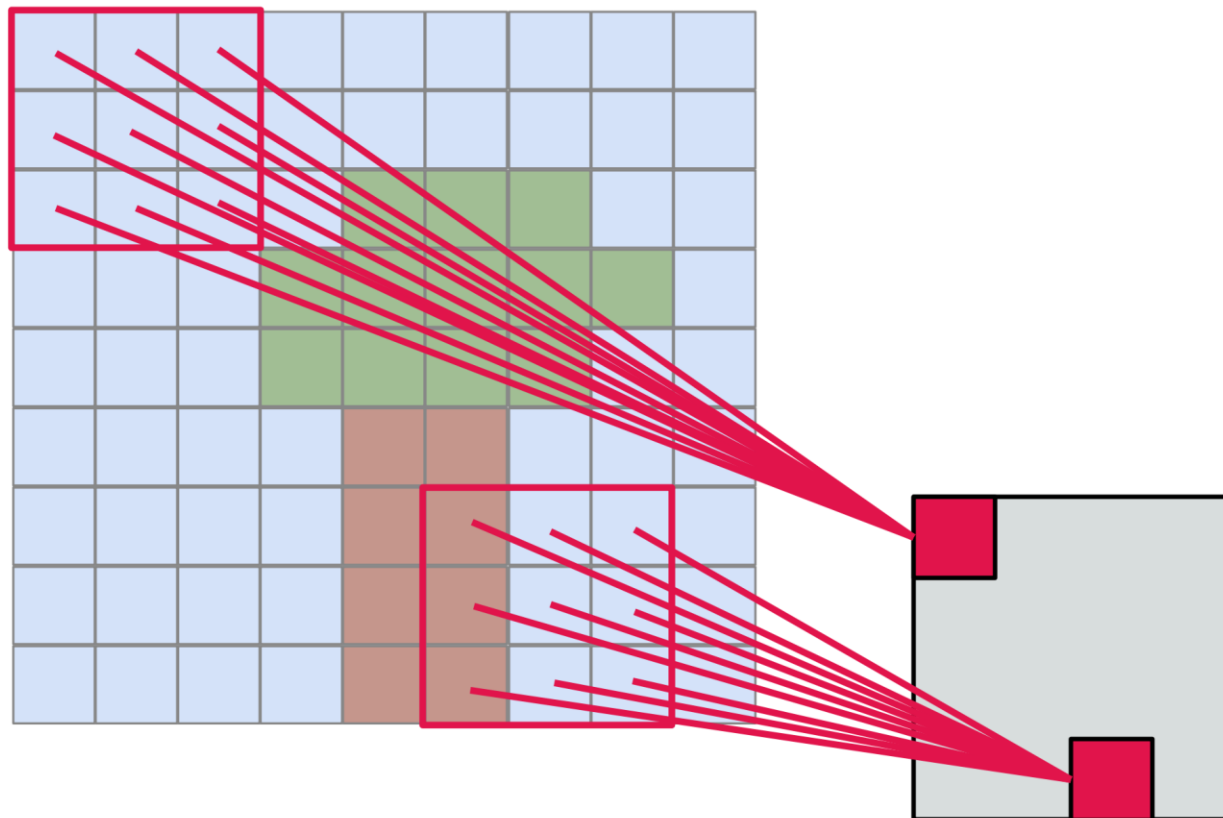


IMAGENET



2 - CONVOLUTIONAL LAYERS

KERNEL

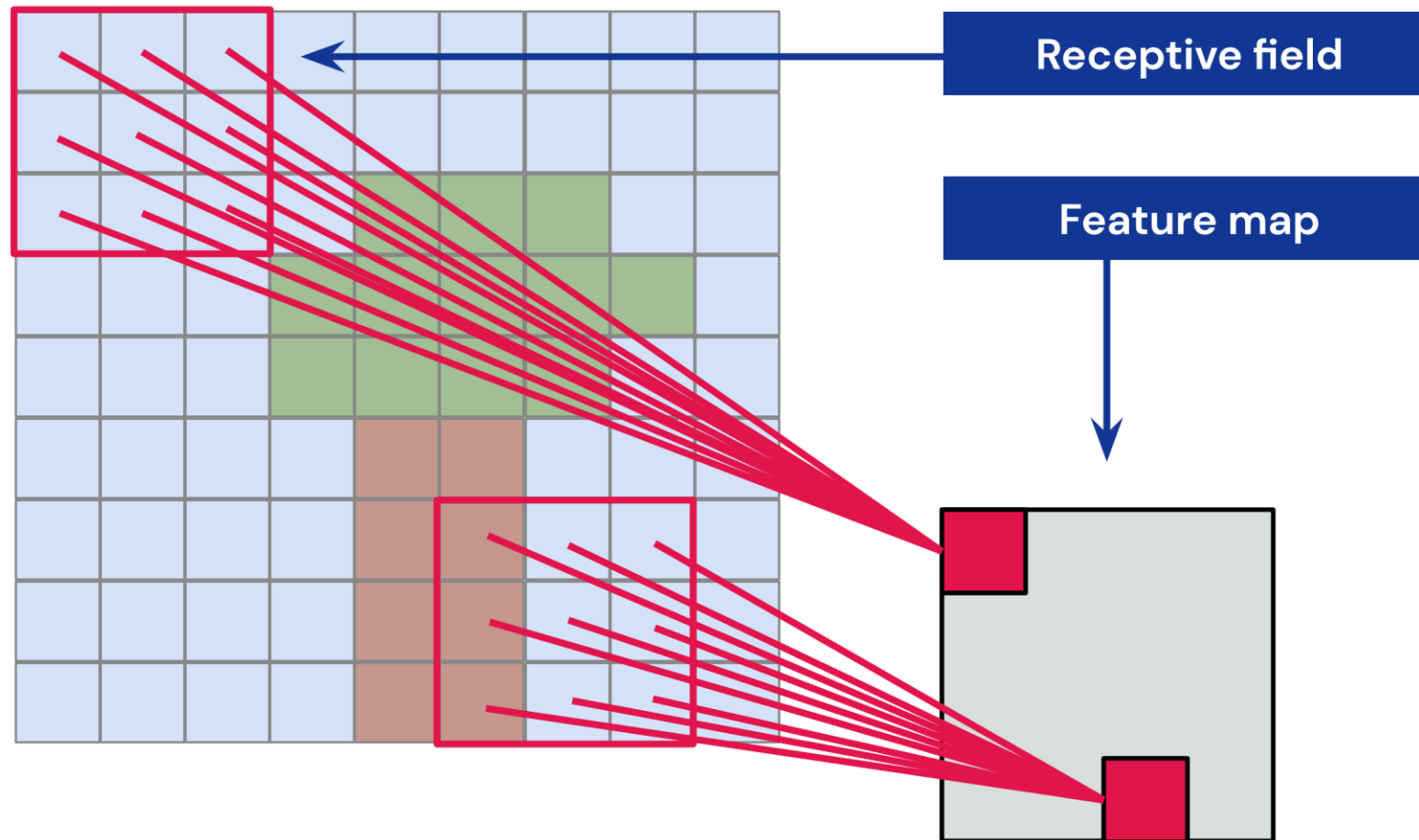


$$y = \mathbf{w} * \mathbf{x} + b$$

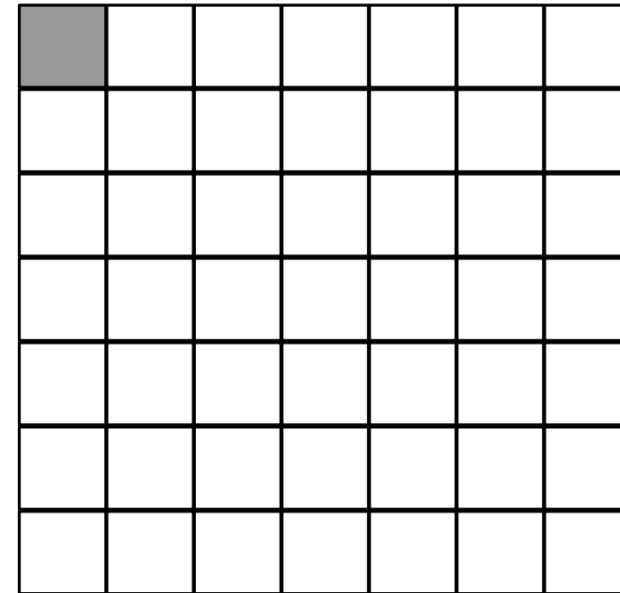
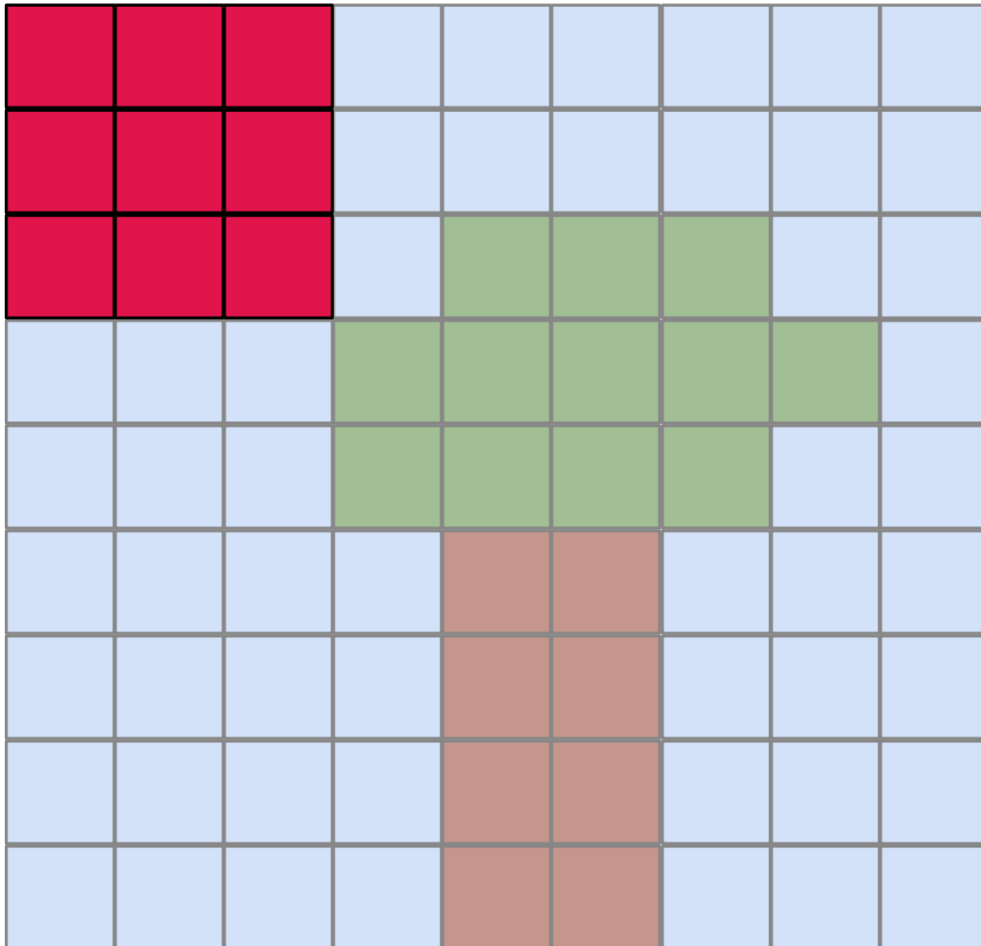
convolutional units
3X3 receptive field



FEATURE MAP & RECEPTIVE FIELD

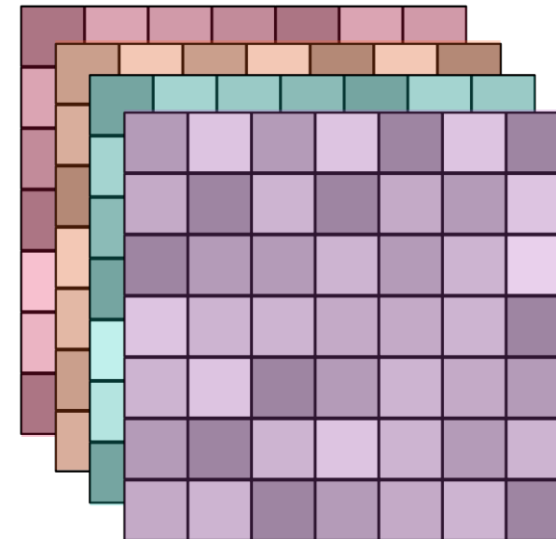
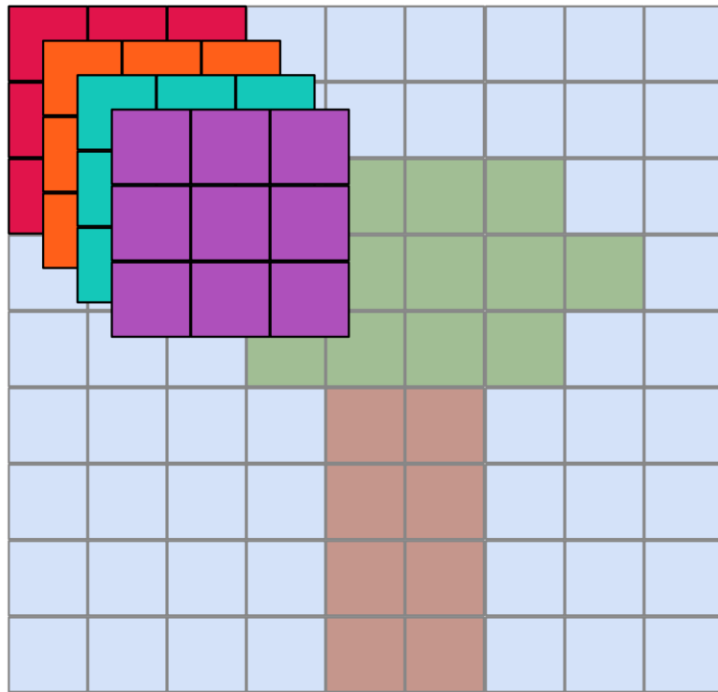


SLIDING ACROSS THE IMAGE



The **kernel** slides across the image and produces an output value at each position

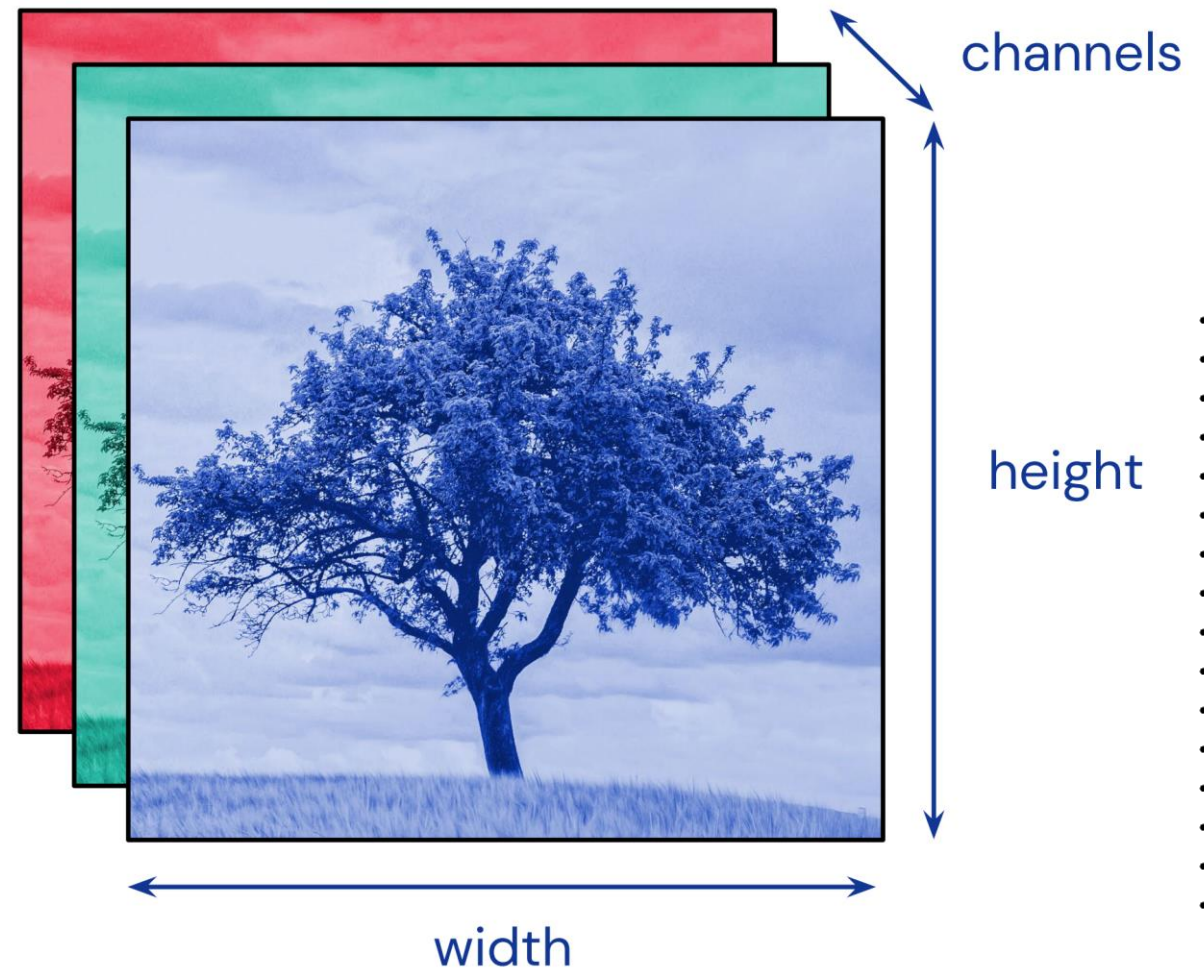
MULTIPLE CHANNELS



We convolve multiple kernels and obtain multiple feature maps or **channels**

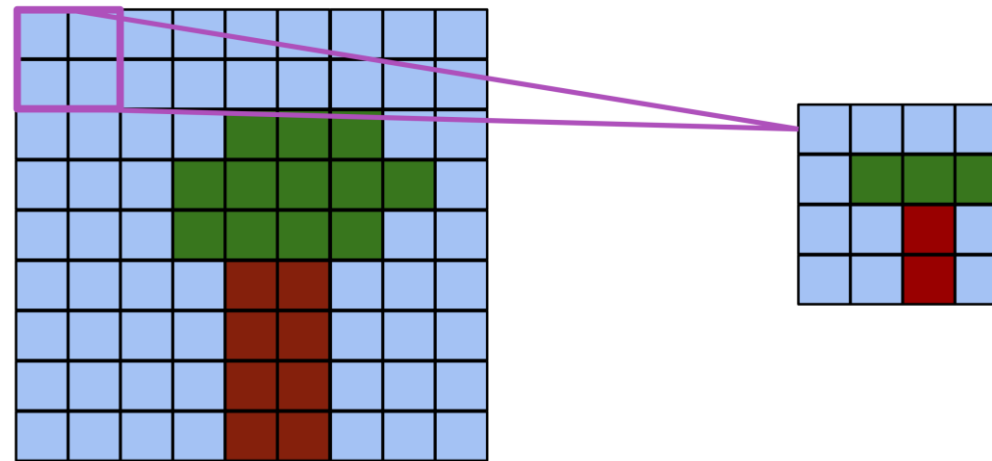
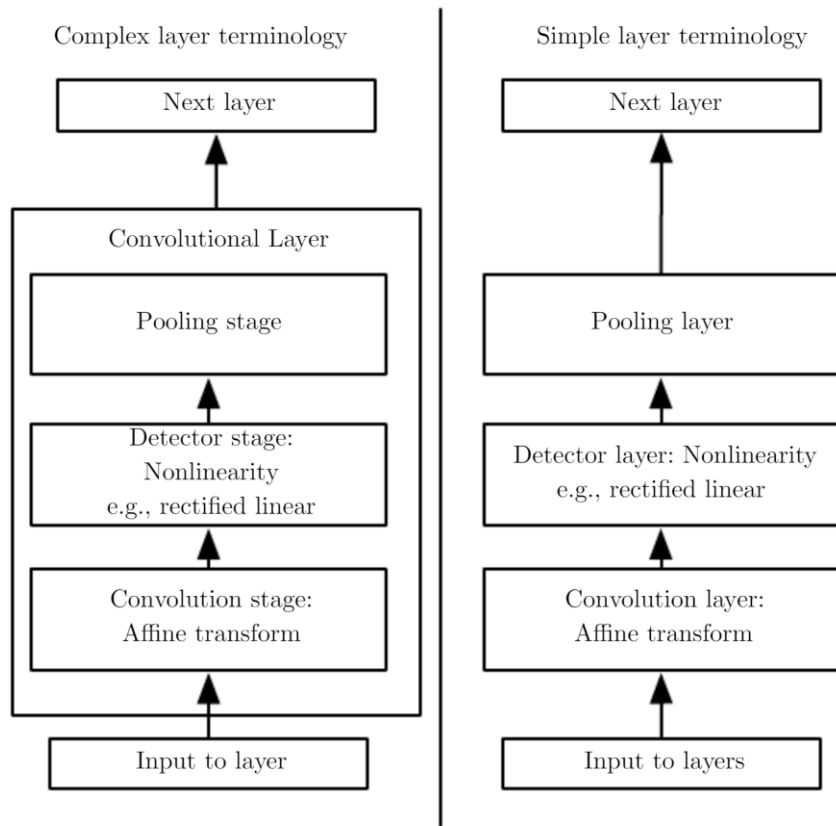


INPUTS AND OUTPUTS ARE TENSORS



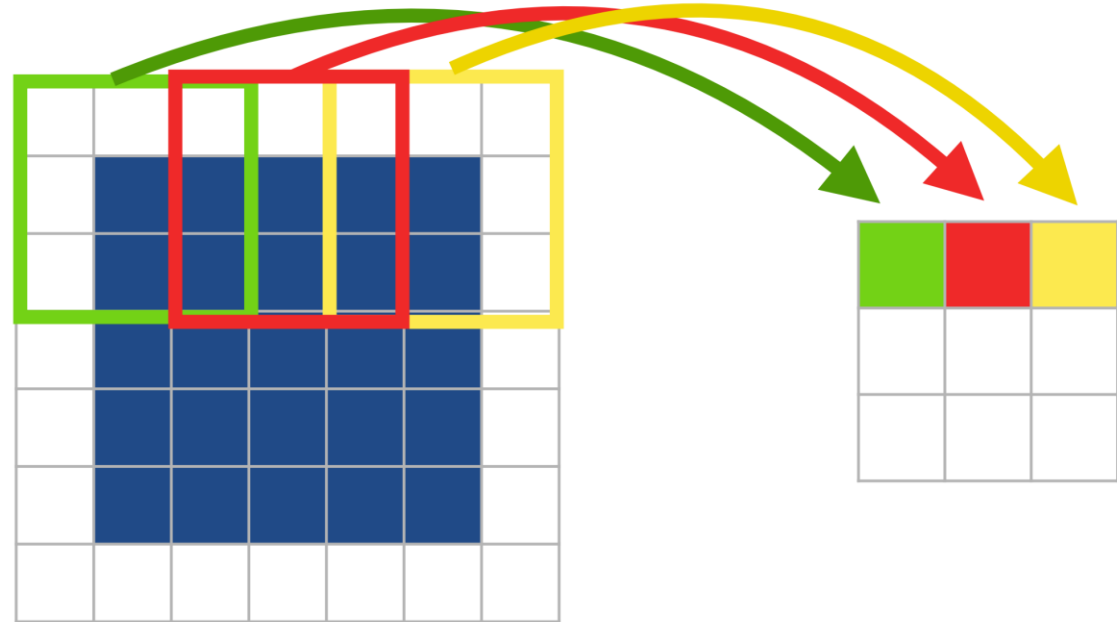
3 - P O O L I N G

WHAT IS POOLING?

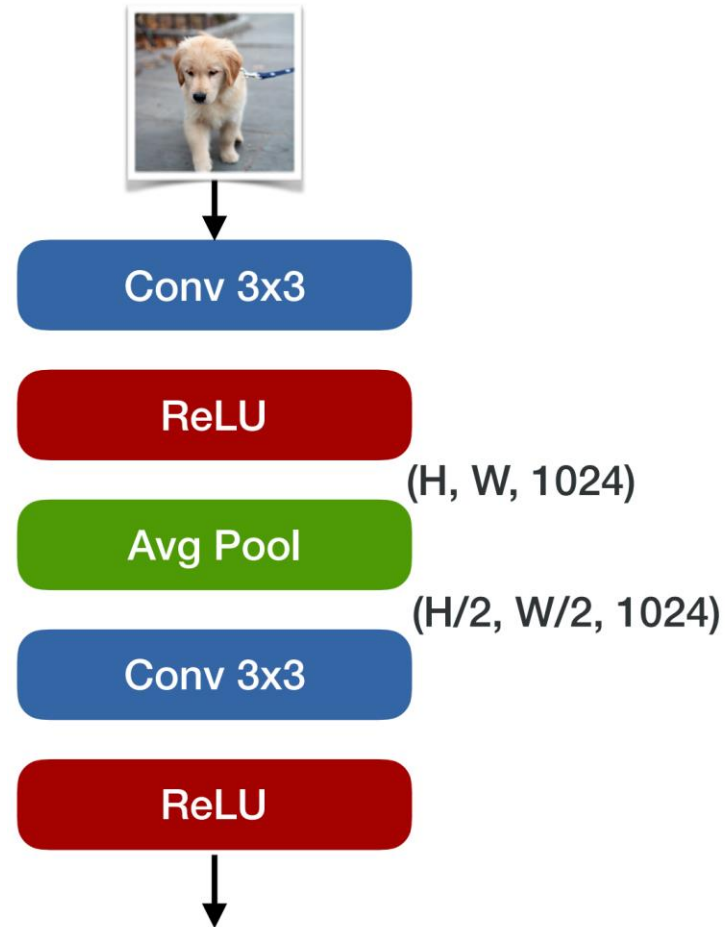


AVERAGE POOLING

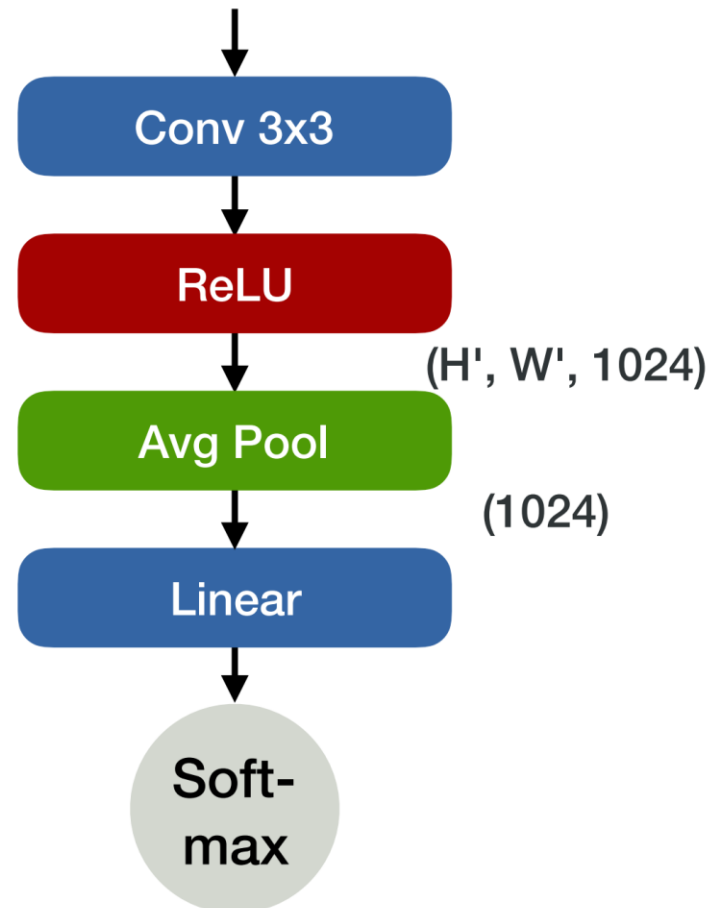
$$f_c(\mathbf{x}) = \text{mean}_{i,j}(\mathbf{x}_{i,j,c})$$



AVERAGE POOLING - REDUCE DIMENSIONALITY

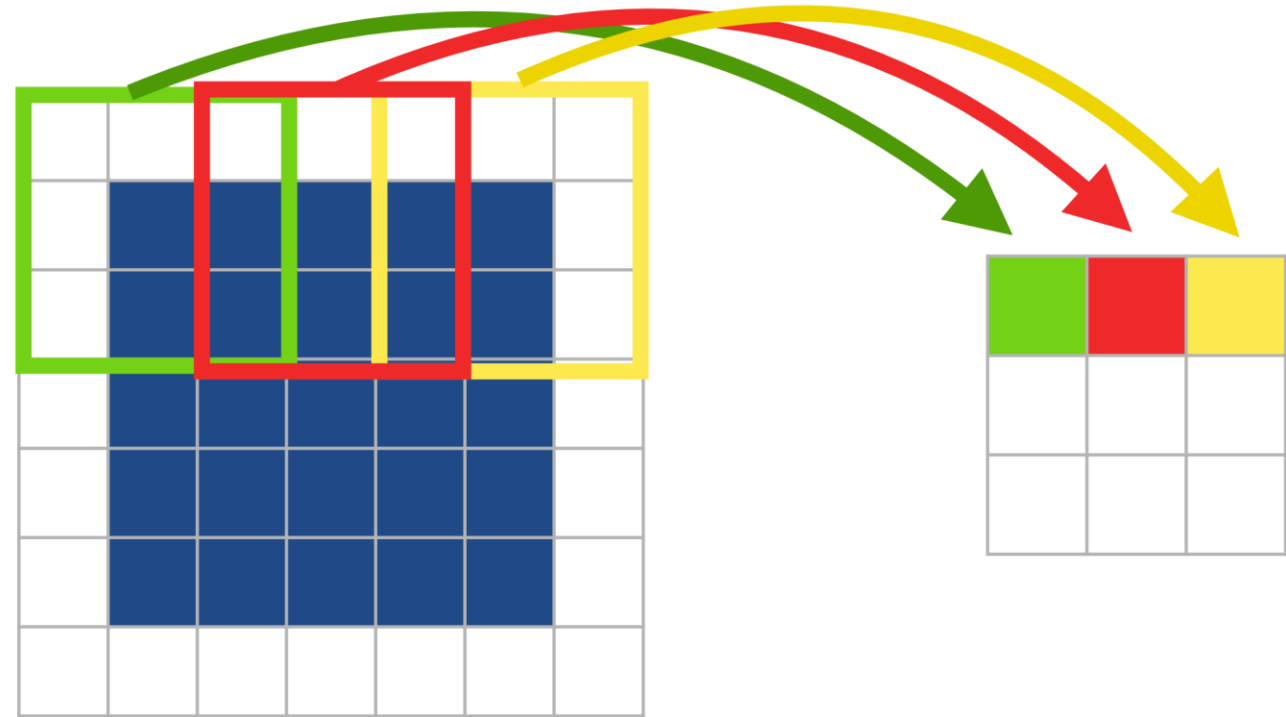


GLOBAL AVERAGE POOLING (GAP)

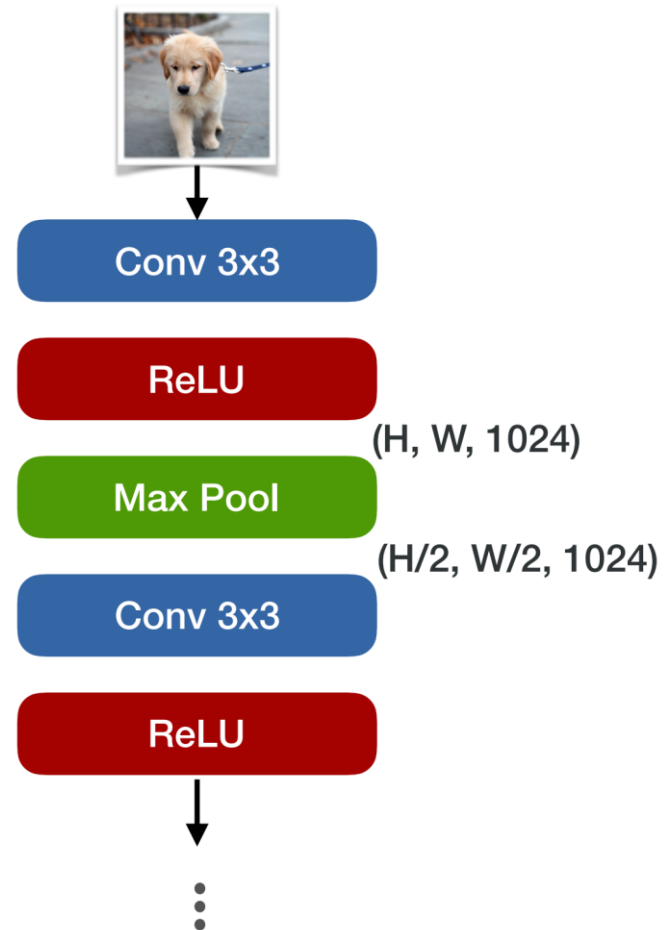


MAX POOLING

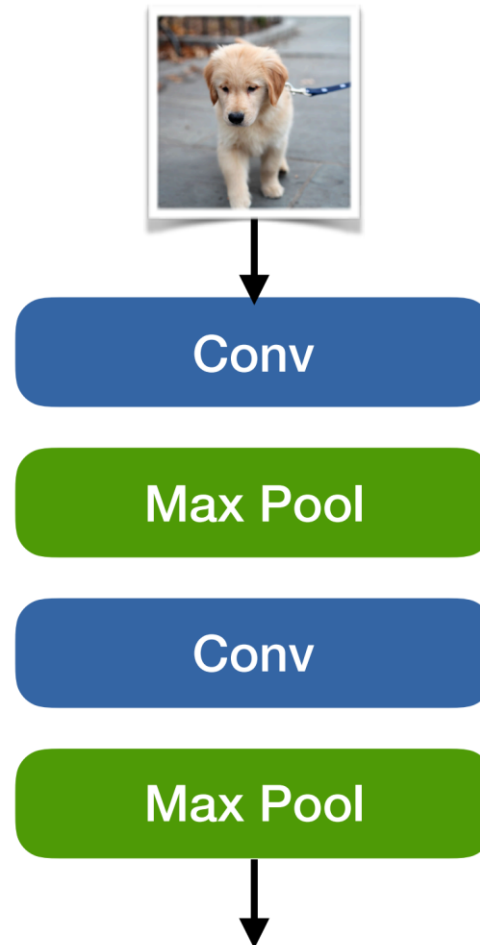
$$f_c(\mathbf{x}) = \max_{i,j}(\mathbf{x}_{i,j,c})$$



MAX POOLING - REDUCE DIMENSIONALITY

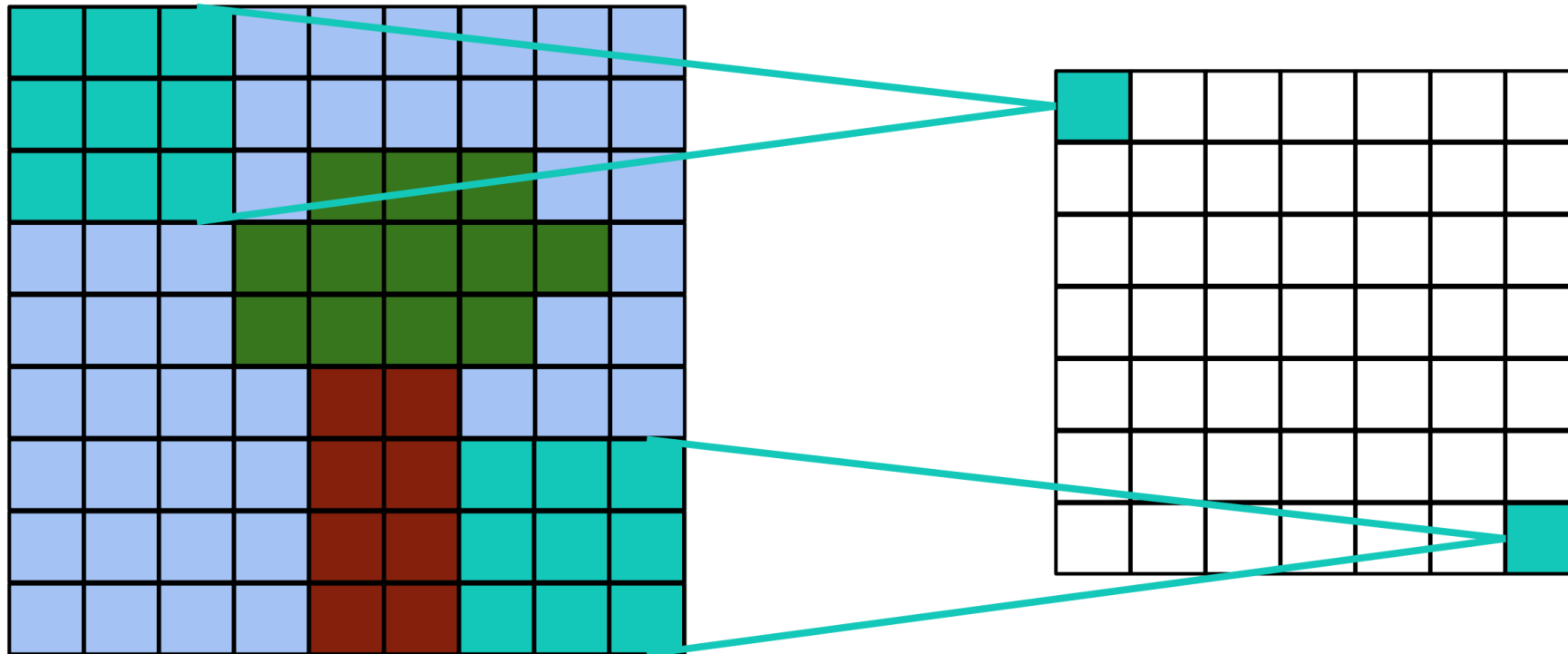


MAX POOLING - NONLINEARITY

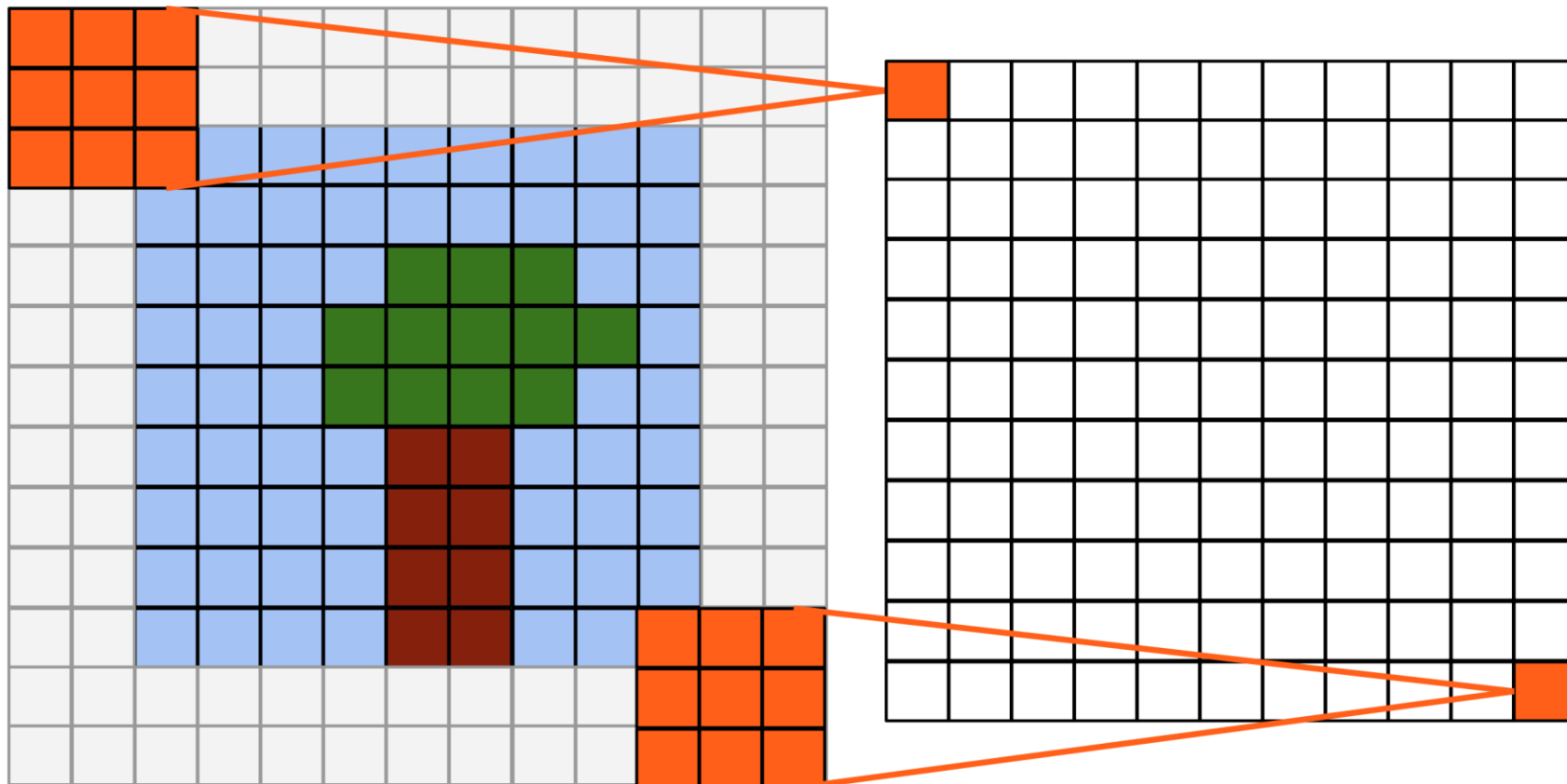


4 - P A D D I N G , S T R I D I N G A N D G R O U P S

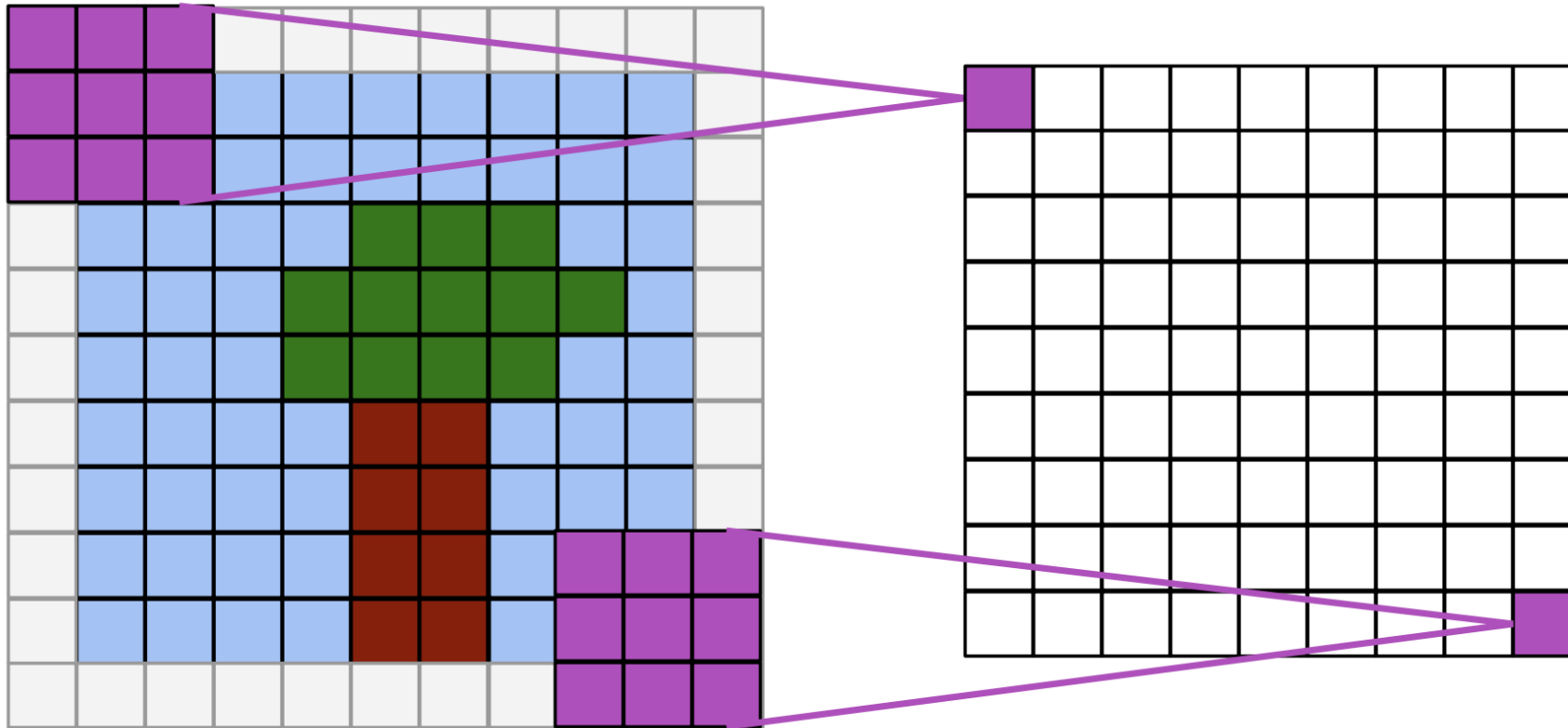
PADDING



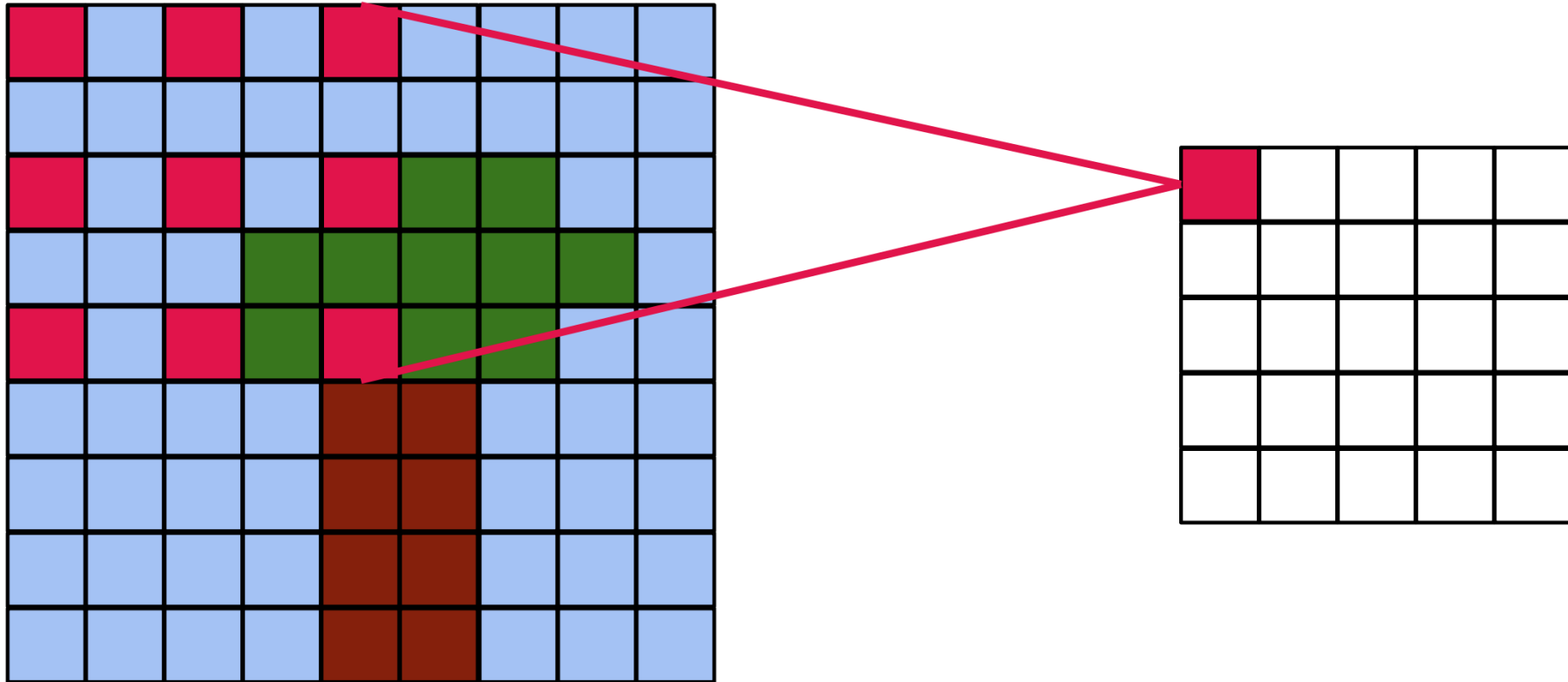
PADDING



PADDING

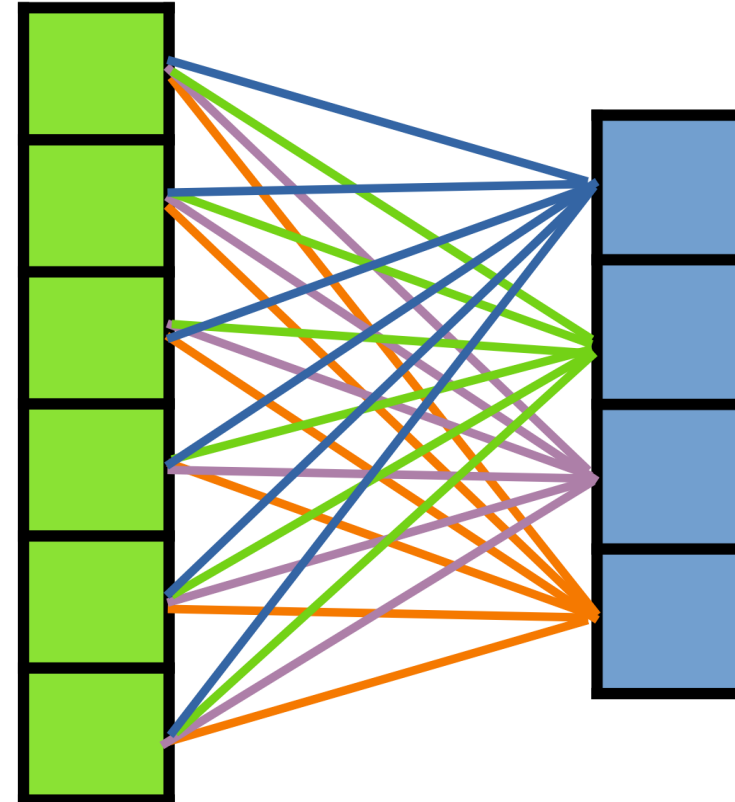


DILATION



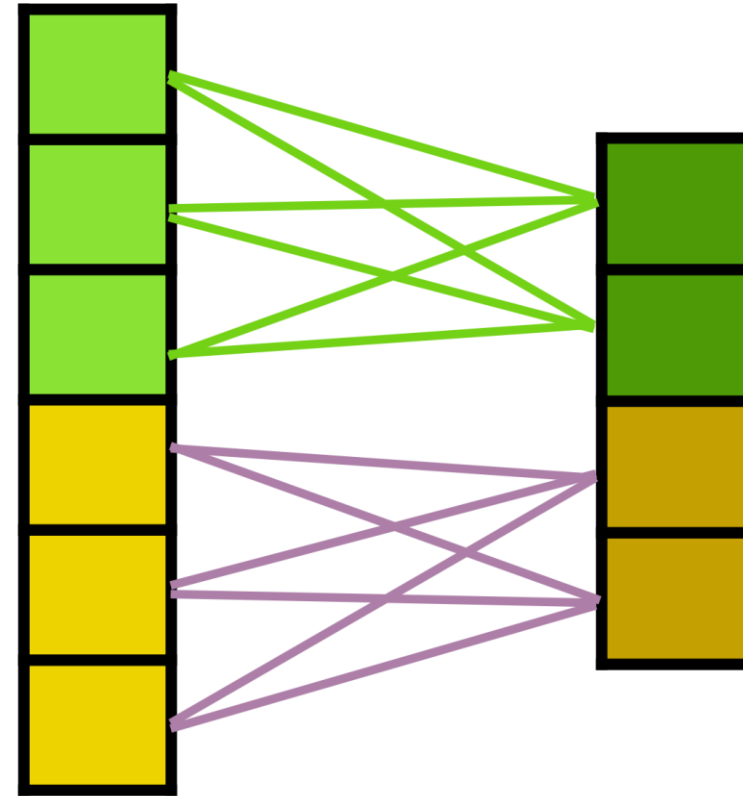
GROUPS

Every input channel is connected to every output channel



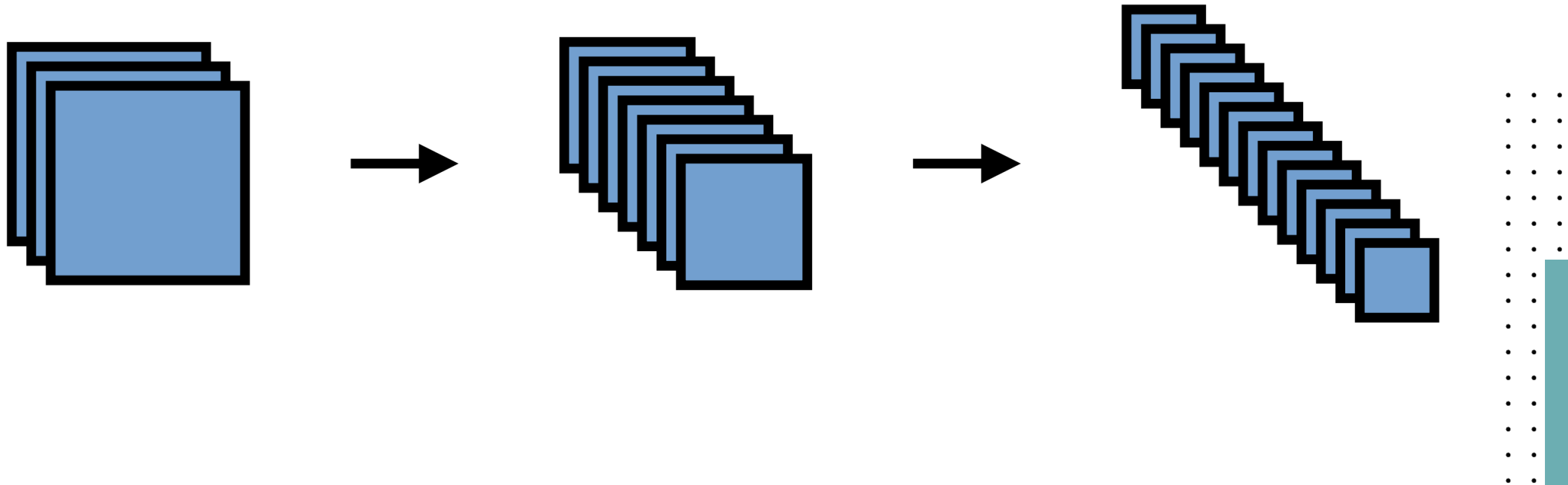
GROUPS

- Split channels into g groups
- Reduce parameters and computation by factor g



5 - DESIGN PRINCIPLES OF CONVOLUTIONAL NETWORKS

USE STRIDING, INCREASE CHANNELS



KEEP KERNELS SMALL



Conv 7x7

Conv 3x3

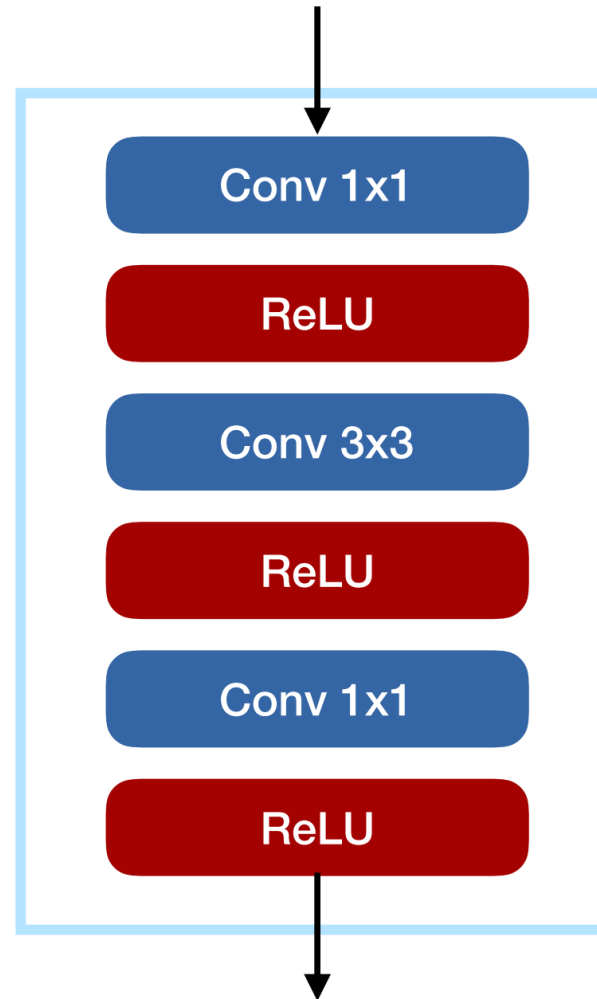
Conv 3x3



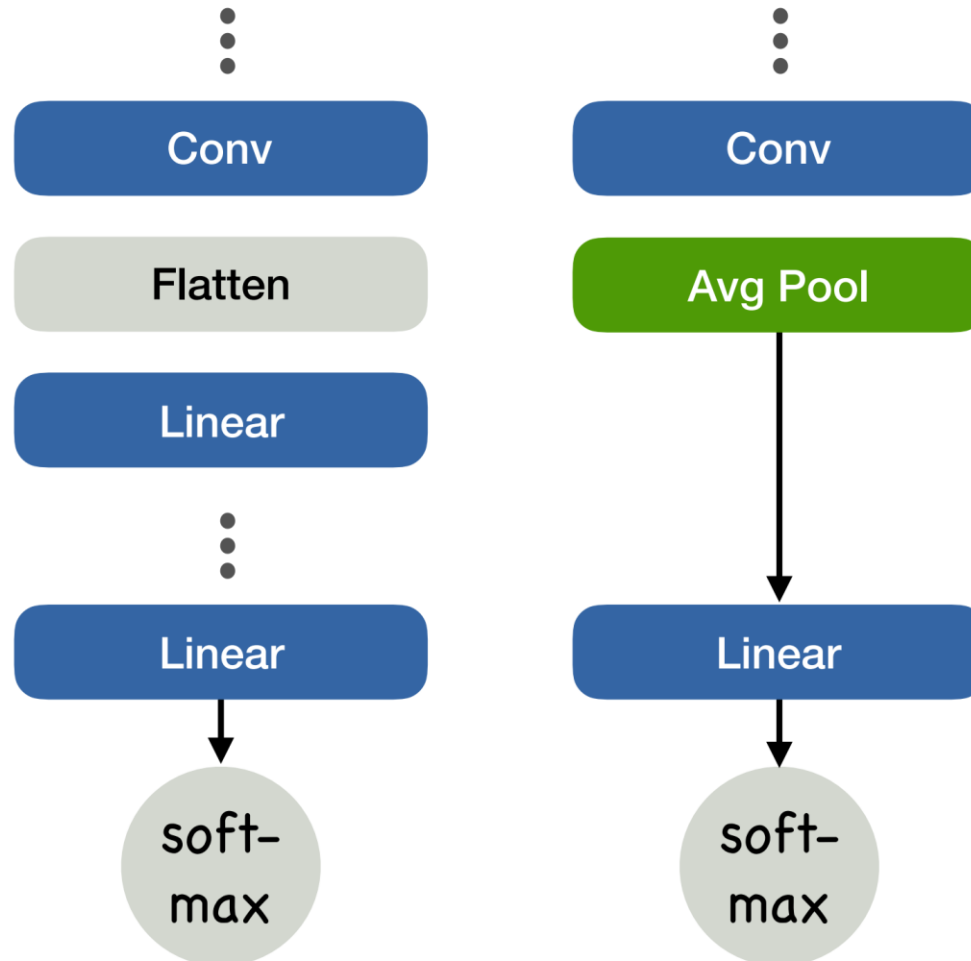
Conv 3x3



REPEAT PATTERNS

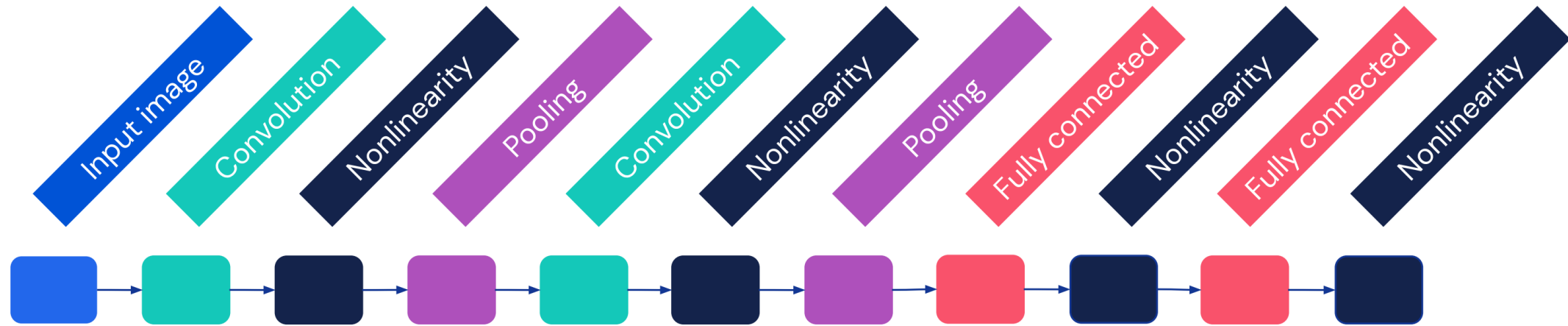


ALL CONVOLUTIONAL

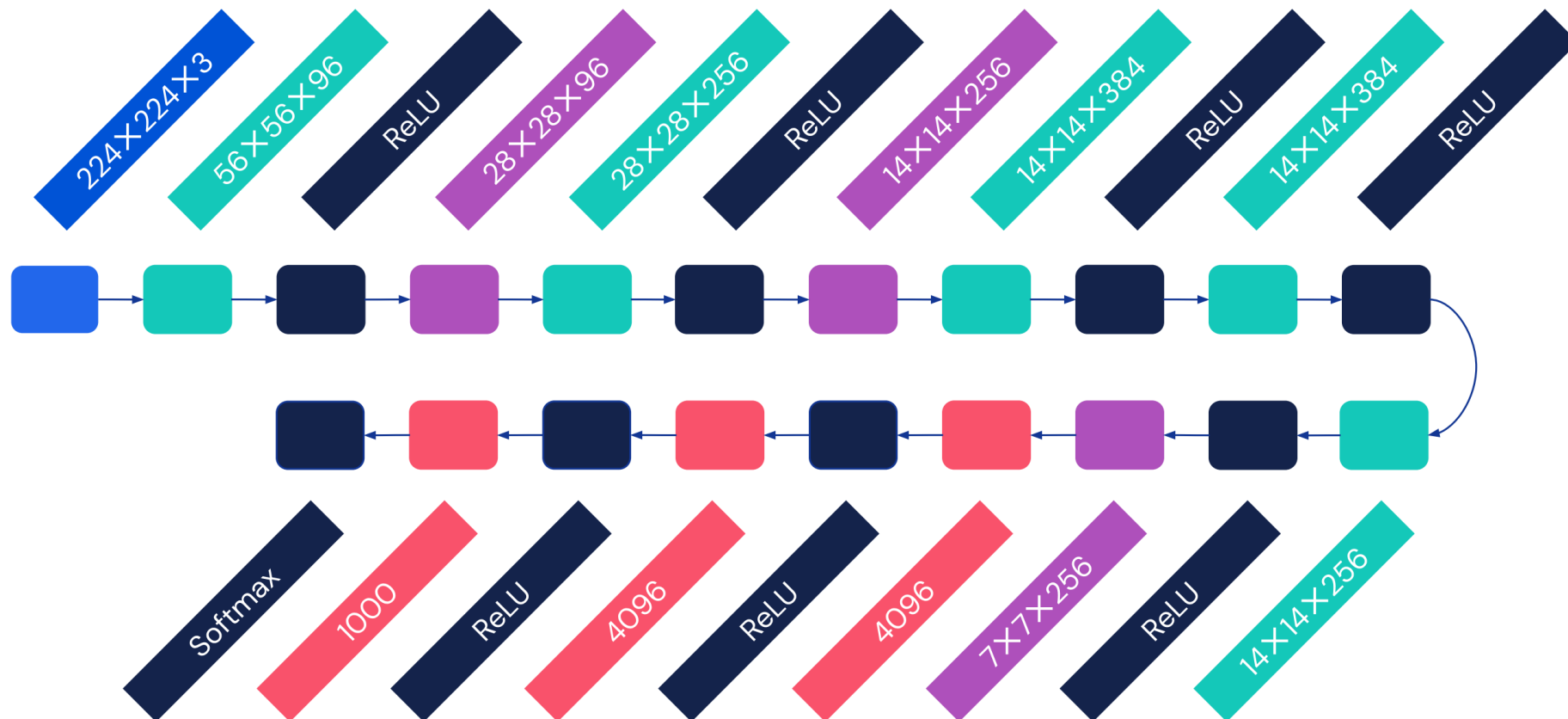


6 - MODEL EXAMPLES

LENET-5

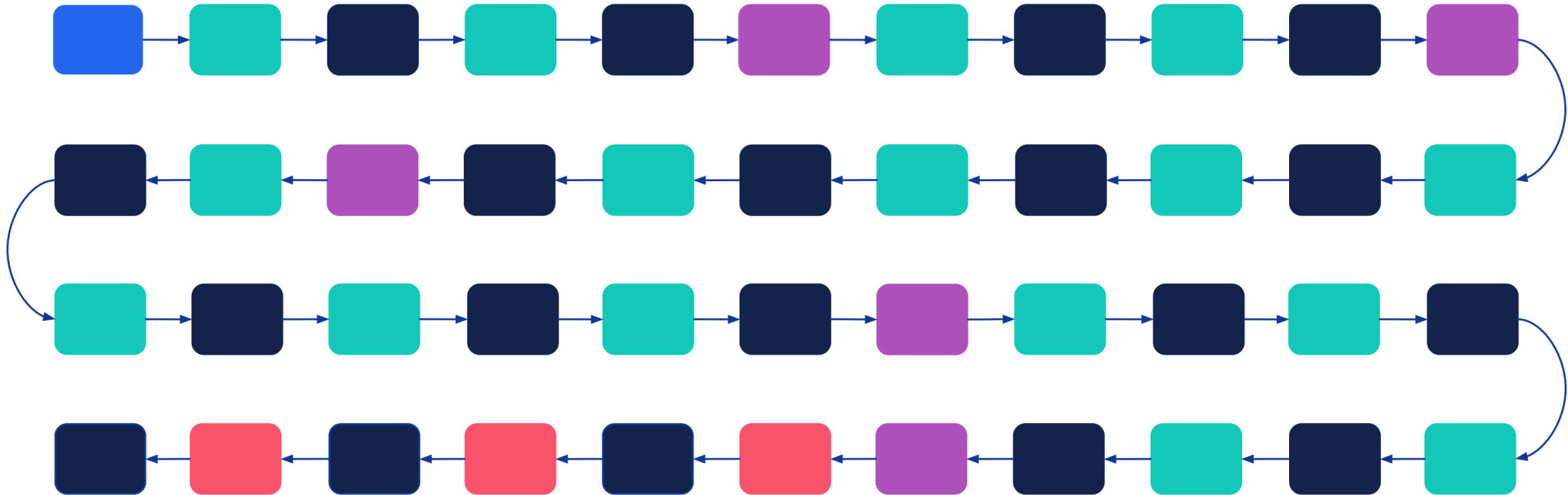


ALEXNET

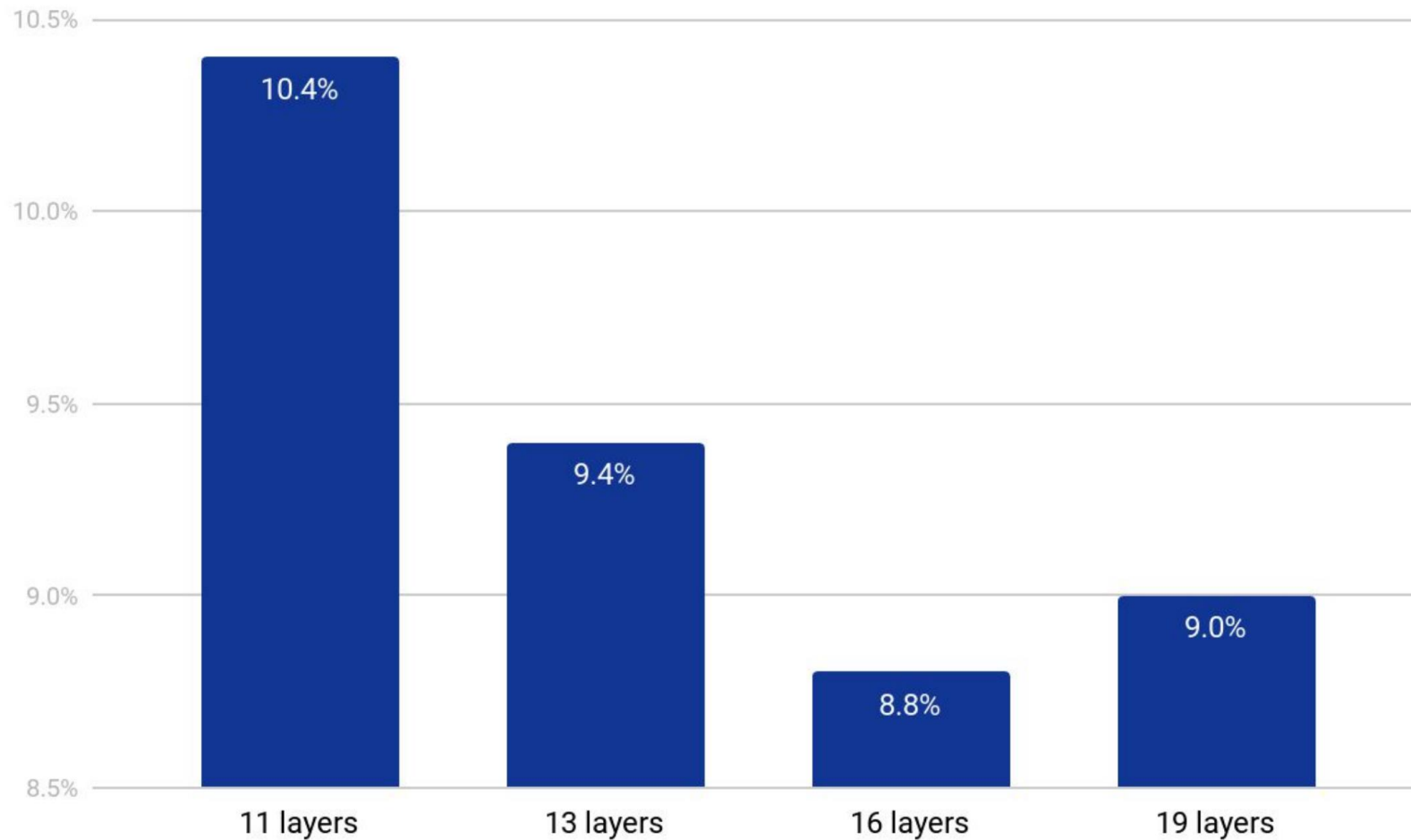


7 - D E P T H

VGGNET (2014): BUILDING VERY DEEP CONVNETS



VGGNET (2014): ERROR PLATEAUS AFTER 16 LAYERS



DATA AUGMENTATION



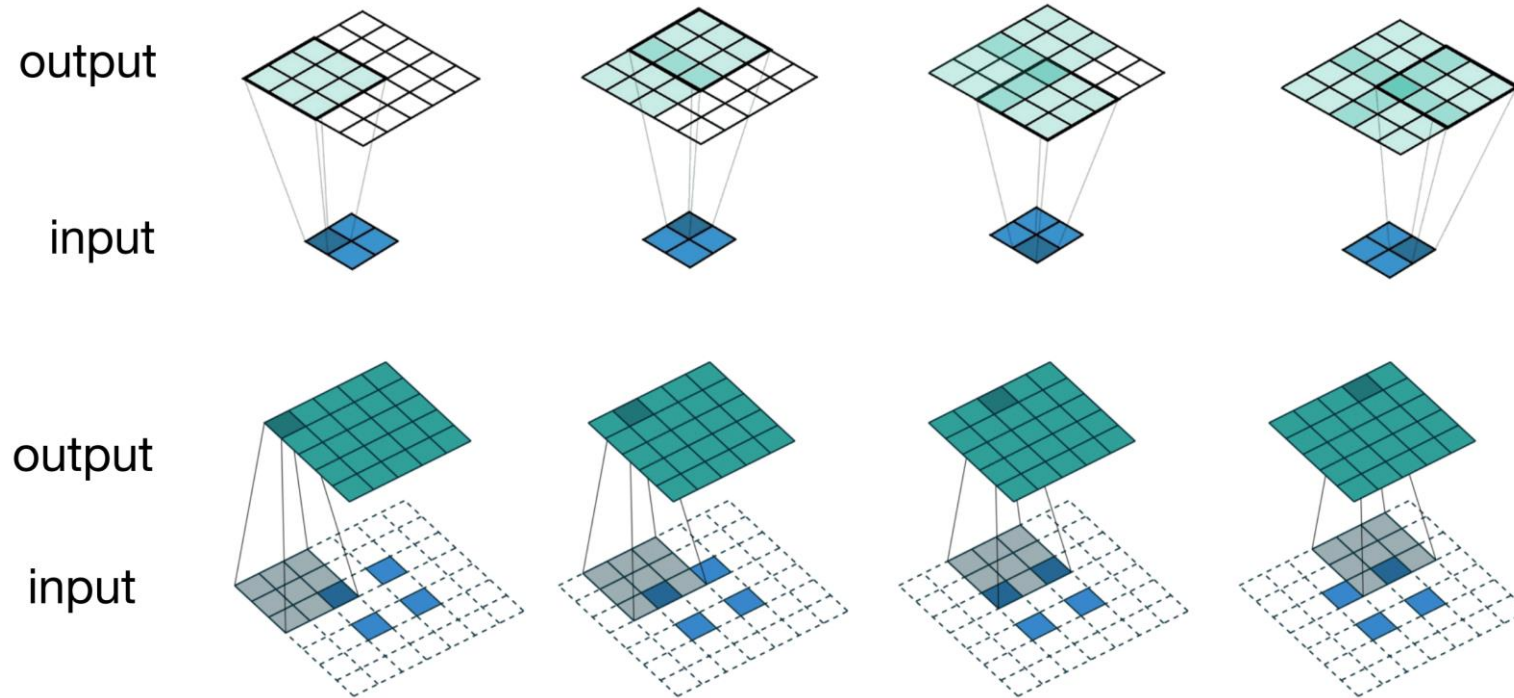
By design, convnets are only robust against **translation**

Data augmentation makes them robust against other transformations: rotation, scaling, shearing, warping, ...



8 - U P - C O N V O L U T I O N S

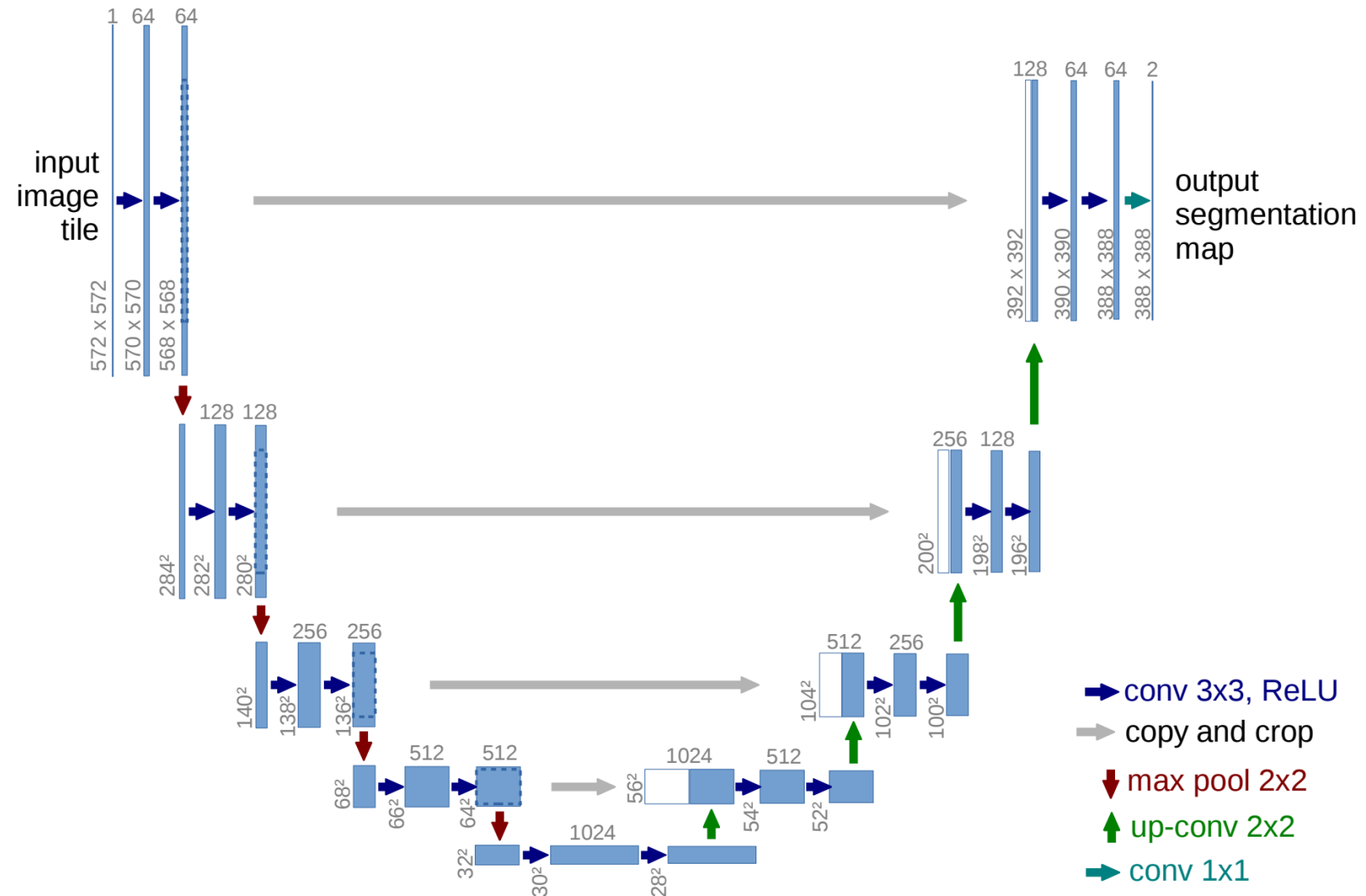
UP-CONVOLUTION



```
model: torch.nn.Module = torch.nn.ConvTranspose2d(16, 33, 3)
input: torch.Tensor = torch.randn(20, 16, 50, 100)
output: torch.Tensor = model(input)
```

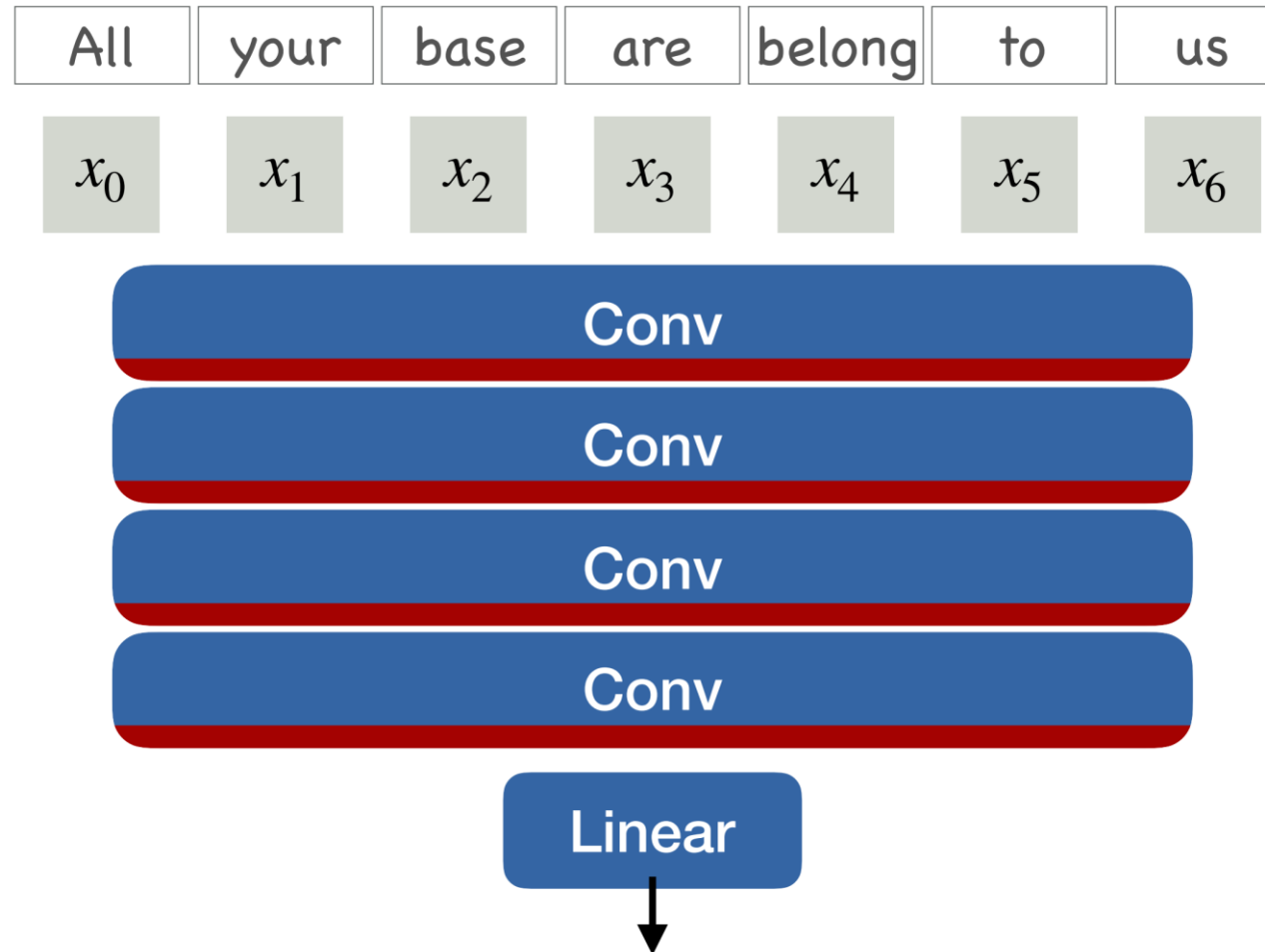


U - NET



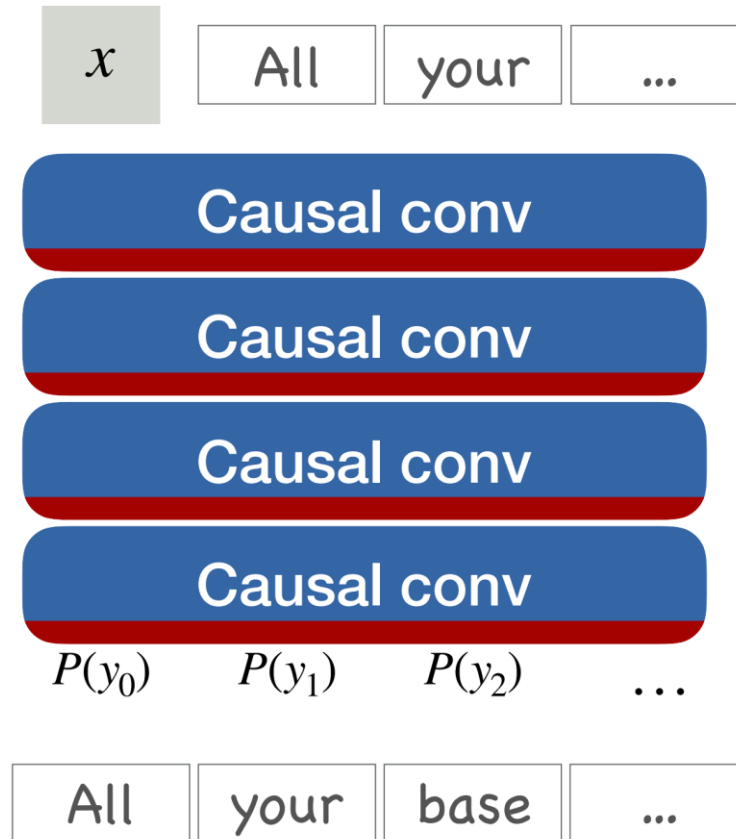
9 - OTHER DIMENSIONS

1D - TEMPORAL CONVOLUTIONS

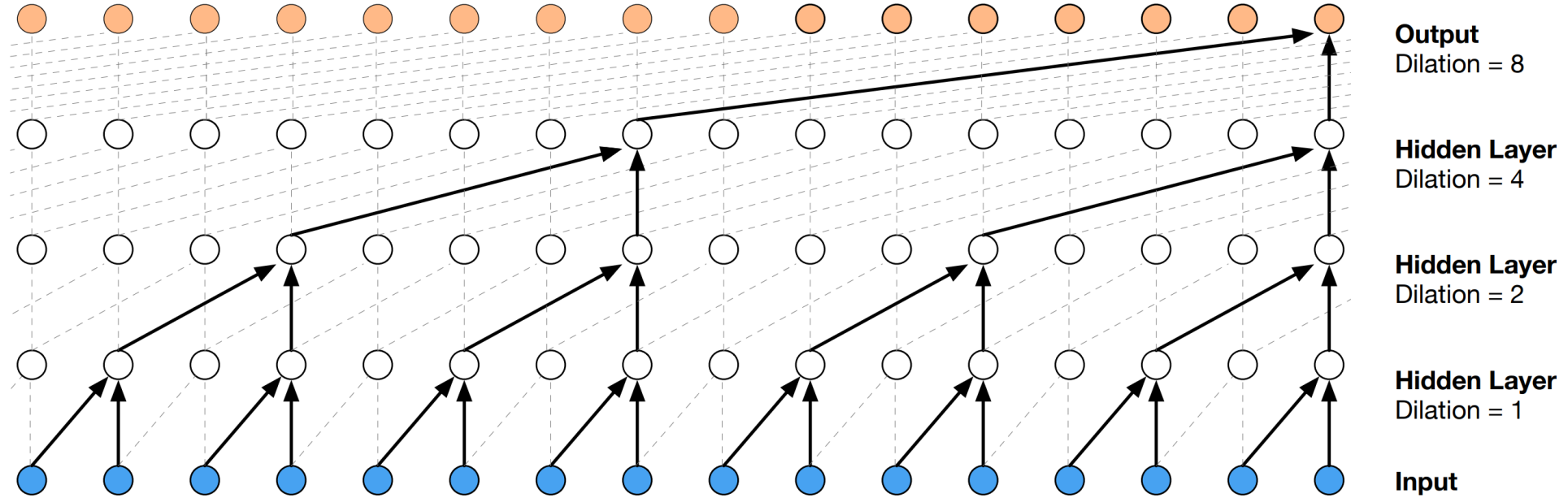


1D - CAUSAL CONVOLUTION

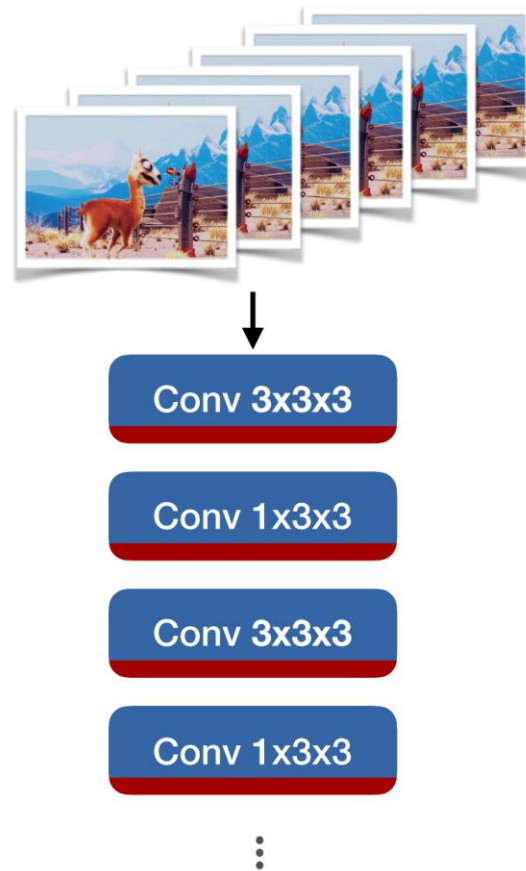
$$\mathbf{Z}_{t,b} = \mathbf{b}_c + \sum_{i=0}^{w-1} \sum_{j=0}^{C_1-1} \mathbf{x}_{t+i-w,b+j} \mathbf{w}_{i,j}$$



1D - WAVENET



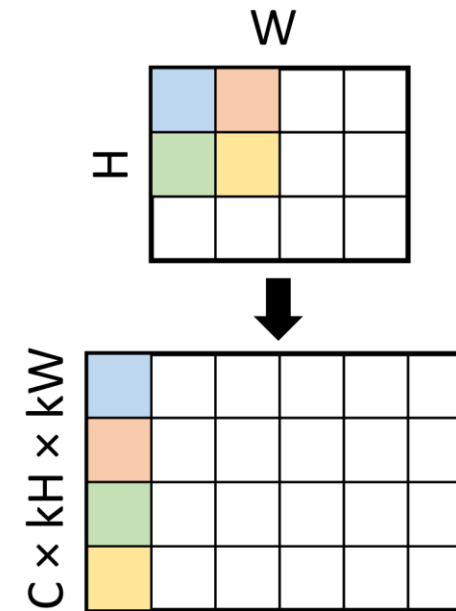
3D - TEMPORAL CONVOLUTION



1 0 - F O L D & U N F O L D

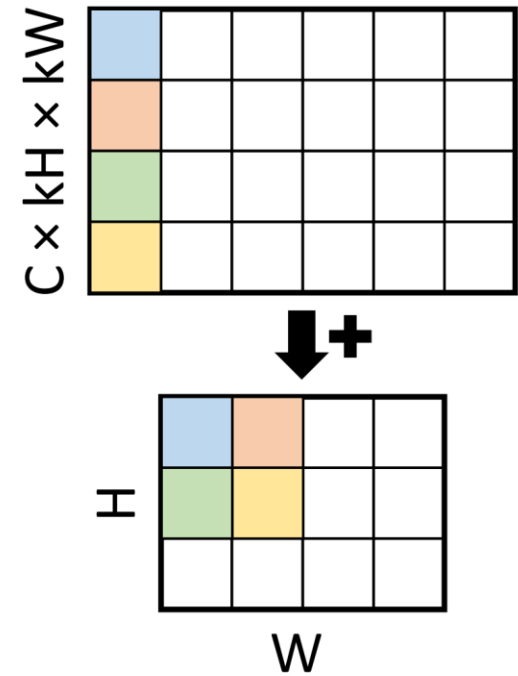
UNFOLD

- Receives a batch of images:
 - Image shape: N, C, H, W
- Extract blocks:
 - Block size: $C \times kH \times kW$
 - Num of block per image: $H' \times W'$
- Return a batch of blocks:
 - Output shape: $N, C \times kH \times kW, H' \times W'$



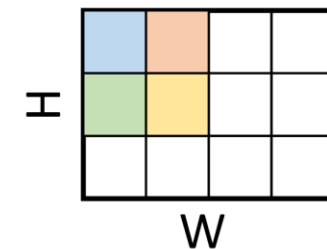
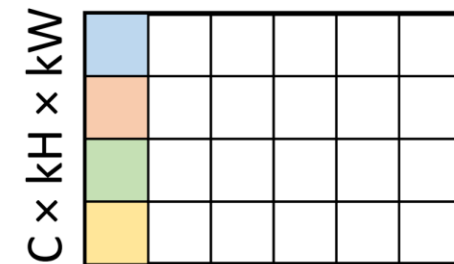
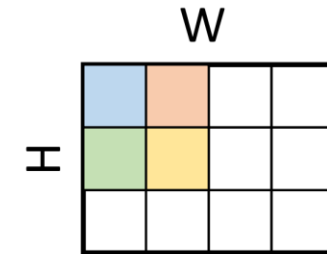
FOLD

- The opposite of `F.unfold`.
- Receives `blocks`.
- Receives `output_size`.
 - Creates an image `output` of that size.
 - Initialize `output` with zeros.
- Iterates over the blocks.
 - Adds each element to its place in `output`.



FOLD & UNFOLD

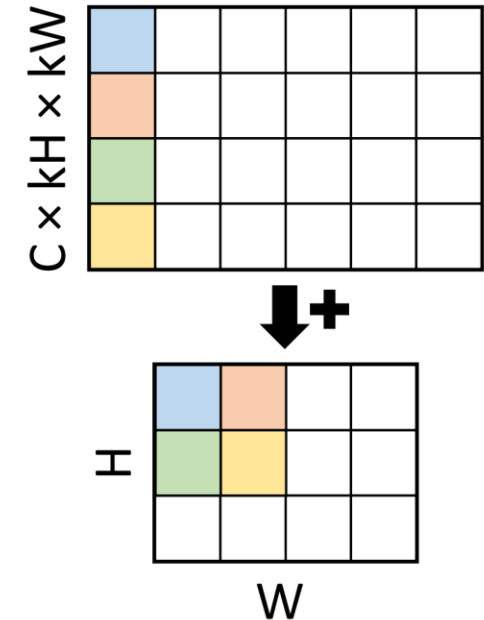
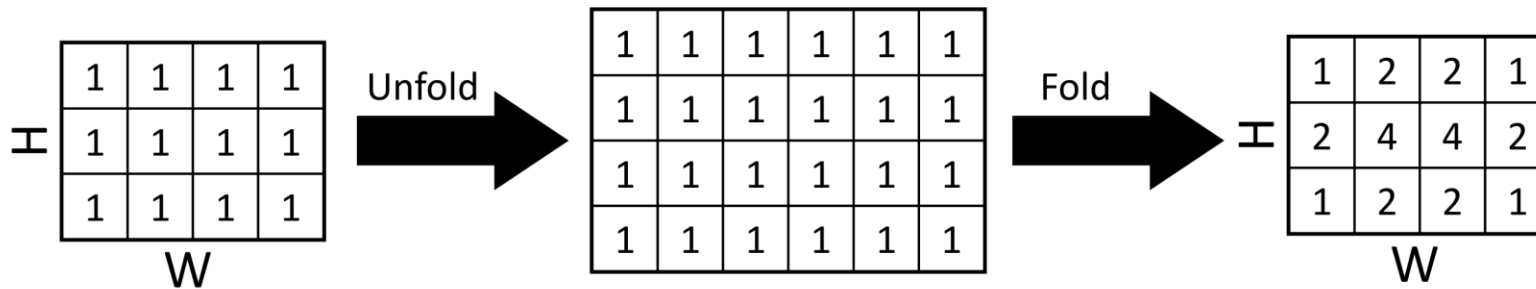
- Extract blocks from an image:
 - `F.unfold` (also called “im2col”).
- Apply an operation on each block:
 - Linear, max pooling, etc.
- Combine blocks into an image:
 - `F.fold` (also called “col2im”).



BLOCKS OVERLAPPING

```
blocks = torch.rand(size=(2, 1 * 2 * 2, 2 * 3))
output_size = (3, 4)

# fold back into an image
output = F.fold(blocks, output_size, kernel_size=2)
```



UNFOLD PARAMETERS

- **kernel_size** (*int* or *tuple*) – the size of the sliding blocks
- **dilation** (*int* or *tuple*, *optional*) – a parameter that controls the stride of elements within the neighborhood. Default: 1
- **padding** (*int* or *tuple*, *optional*) – implicit zero padding to be added on both sides of input. Default: 0
- **stride** (*int* or *tuple*, *optional*) – the stride of the sliding blocks in the input spatial dimensions. Default: 1



FOLD PARAMETERS

- **output_size** (*int* or *tuple*) – the shape of the spatial dimensions of the output (i.e., `output.sizes()[2:]`)
- **kernel_size** (*int* or *tuple*) – the size of the sliding blocks
- **dilation** (*int* or *tuple*, *optional*) – a parameter that controls the stride of elements within the neighborhood. Default: 1
- **padding** (*int* or *tuple*, *optional*) – implicit zero padding to be added on both sides of input. Default: 0
- **stride** (*int* or *tuple*) – the stride of the sliding blocks in the input spatial dimensions. Default: 1



1 1 - G A T H E R & S C A T T E R

GATHER

torch.gather

`torch.gather(input, dim, index, *, sparse_grad=False, out=None) → Tensor`

Gathers values along an axis specified by *dim*.

For a 3-D tensor the output is specified by:

```
out[i][j][k] = input[index[i][j][k]][j][k] # if dim == 0
out[i][j][k] = input[i][index[i][j][k]][k] # if dim == 1
out[i][j][k] = input[i][j][index[i][j][k]] # if dim == 2
```

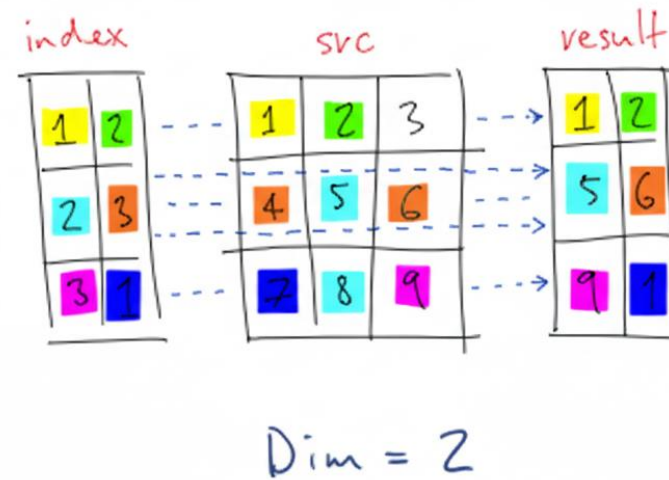
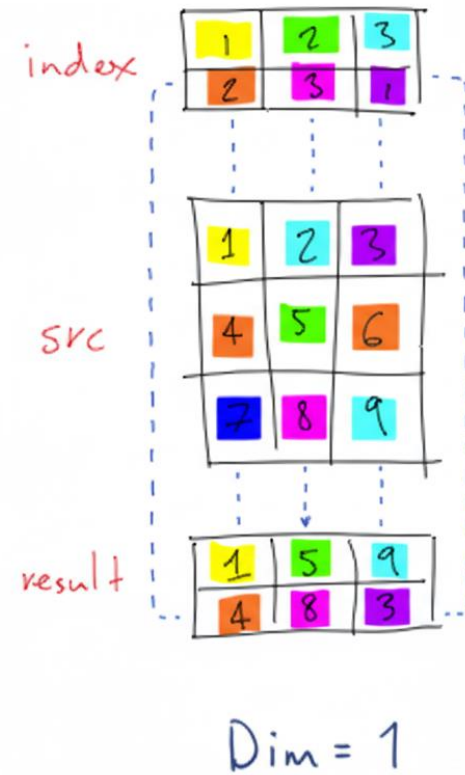
`input` and `index` must have the same number of dimensions. It is also required that `index.size(d) <= input.size(d)` for all dimensions `d != dim`. `out` will have the same shape as `index`. Note that `input` and `index` do not broadcast against each other. When `index` is empty, we always return an empty output with the same shape without further error checking.

Parameters:

- **input** ([Tensor](#)) – the source tensor
- **dim** ([int](#)) – the axis along which to index
- **index** ([LongTensor](#)) – the indices of elements to gather



GATHER INDEXES



GATHER INDEXES - 2D EXAMPLE

src_2d =

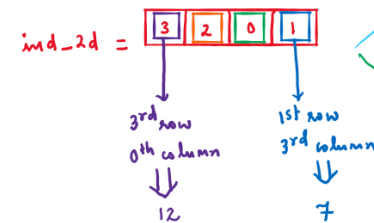
0	1	2	3
4	5	6	7
8	9	10	11
12	13	14	15

Dims for case of 2D

When dim = 0, we choose rows.

When dim = 1, we choose columns.

for dim = 0



out_2d =

12	9	2	7
----	---	---	---

① Tells shape of the output-tensor.

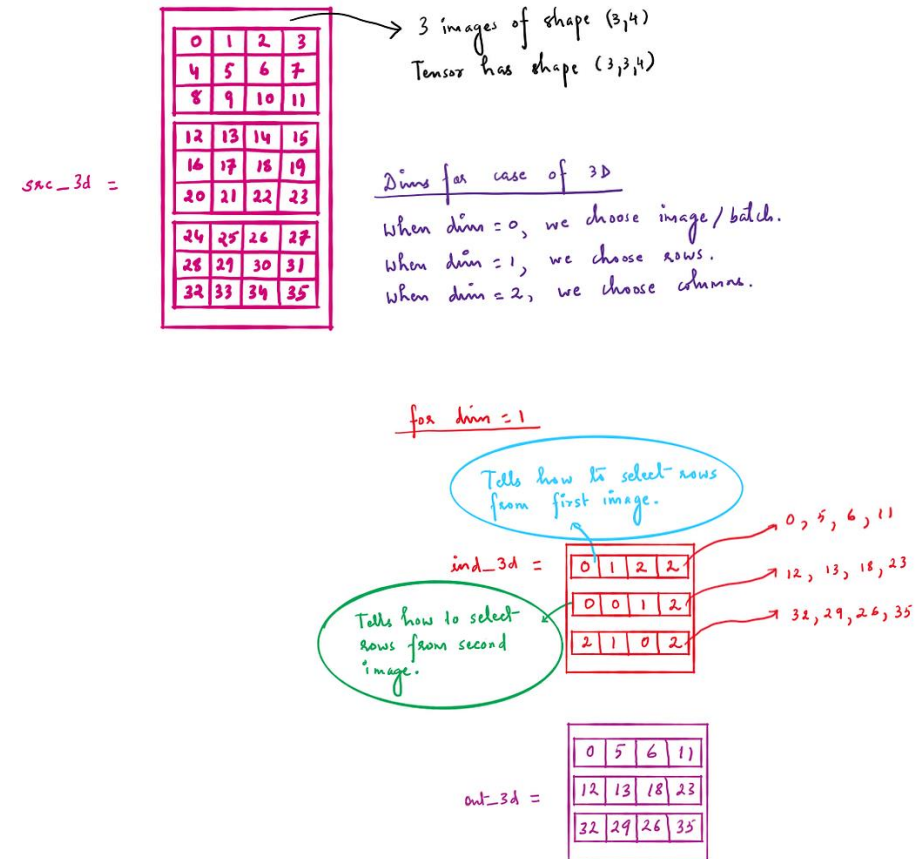
② Tells which rows to choose.

① Shape same as ind_2d, i.e. (1,4)

② Elements filled a/c the process described



GATHER INDEXES - 3D EXAMPLE



SCATTER

torch.Tensor.scatter_

`Tensor.scatter_(dim, index, src, *, reduce=None) → Tensor #`

Writes all values from the tensor `src` into `self` at the indices specified in the `index` tensor. For each value in `src`, its output index is specified by its index in `src` for `dimension != dim` and by the corresponding value in `index` for `dimension = dim`.

For a 3-D tensor, `self` is updated as:

```
self[index[i][j][k]][j][k] = src[i][j][k] # if dim == 0
self[i][index[i][j][k]][k] = src[i][j][k] # if dim == 1
self[i][j][index[i][j][k]] = src[i][j][k] # if dim == 2
```

This is the reverse operation of the manner described in `gather()`.

It is also required that `index.size(d) <= src.size(d)` for all dimensions `d`, and that `index.size(d) <= self.size(d)` for all dimensions `d != dim`. Note that `input` and `index` do not broadcast against each other for NPU, so when running on NPU, `input` and `index` must have the same number of dimensions. Standard broadcasting occurs in all other cases.

Moreover, as for `gather()`, the values of `index` must be between `0` and `self.size(dim) - 1` inclusive.



REFERENCES

- <https://www.philkr.net/cs342/material>
- https://storage.googleapis.com/deepmind-media/UCLxDeepMind_2020/L3%20-%20UCLxDeepMind%20DL2020.pdf
- <https://arxiv.org/abs/1505.04597>
- <https://arxiv.org/abs/1609.03499v2>
- https://dl4cv.github.io/DL4CV_Winter24/index.html

